



Using Internet Utility Classes

Version 2026.1
2026-04-20

Using Internet Utility Classes

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Sending an HTTP Request	1
1.1 Introduction to HTTP Requests	1
1.2 HTTP Authentication	2
1.2.1 Authenticating a Request When Using HTTP 1.0	2
1.2.2 Authenticating a Request When Using HTTP 1.1	2
1.2.3 Specifying the Authorization Header Directly	3
1.2.4 Enabling Logging for HTTP Authentication	3
1.3 Specifying Other HTTP Request Properties	4
1.3.1 The Location Property	4
1.3.2 Specifying the Internet Media Type and Character Encoding	4
1.3.3 Using a Proxy Server	5
1.3.4 Using SSL to Connect	5
1.3.5 The HTTPVersion, Timeout, WriteTimeout, and FollowRedirect Properties	6
1.3.6 Handling Chunked Responses	6
1.3.7 Specifying Default Values for HTTP Requests	7
1.4 Setting and Getting HTTP Headers	7
1.5 Managing Keep-Alive Behavior	8
1.6 Handling HTTP Request Parameters	8
1.7 Including a Request Body	9
1.7.1 Sending a Chunked Request	10
1.8 Sending Form Data	11
1.8.1 Example 1	12
1.8.2 Example 2	12
1.9 Inserting, Listing, and Deleting Cookies	12
1.10 Sending the HTTP Request	13
1.11 Creating and Sending Multipart POST Requests	15
1.12 Accessing the HTTP Response	15
1.13 See Also	15
2 Using an HTTP Response	17
2.1 Accessing the Data of the Response	17
2.2 Accessing the HTTP Headers	18
2.3 Accessing Other Information about the Response	18
2.4 See Also	19
3 Email Support in InterSystems IRIS	21
3.1 Supported Email Protocols	21
3.2 How InterSystems IRIS Represents MIME Email Messages	21
3.3 Email Support in Interoperability Productions	22
3.4 See Also	22
4 Creating Email Messages	23
4.1 Creating Single-Part Email Messages	23
4.2 Creating Multipart Email Messages	24
4.3 Specifying Email Message Headers	25
4.3.1 Specifying Basic Email Headers	25
4.3.2 Content-Type Header	25
4.3.3 Content-Transfer-Encoding Header	25
4.3.4 Custom Headers	26

4.4 Adding Attachments to a Message	26
4.4.1 Example: MessageWithAttachment()	27
4.5 Encoding and Character Translation	28
4.6 See Also	28
5 Sending Email via SMTP	29
5.1 Overview	29
5.2 Authenticating to the SMTP Server	29
5.2.1 Enabling XOAUTH2 Authentication	30
5.3 Sending Email	30
5.4 Other Properties of %Net.SMTP	31
5.5 See Also	31
6 Retrieving Email via POP3	33
6.1 Overview	33
6.2 Connecting to the Server	33
6.2.1 Enabling XOAUTH2 Authentication	34
6.3 Getting Information about the Mailbox	35
6.4 Retrieving Messages	36
6.4.1 Encoding and Character Translation of Messages	36
6.5 Retrieving Attachments	36
6.6 Retrieving Attached Messages	37
6.7 Deleting Messages	37
6.8 Updating the Mailbox	38
6.9 Summary of Methods	38
6.10 See Also	40
7 Working with Received Email	41
7.1 Message Basics	41
7.2 Message Contents	41
7.3 Message Headers	42
7.4 Other Message Information	42
7.5 Example 1: ShowMsgInfo()	43
7.6 Example 2: ShowMsgPartInfo()	43
7.7 Example 3: ShowMsgHeaders()	44
7.8 See Also	44
8 Creating, Writing, and Reading MIME Messages	45
8.1 Overview	45
8.2 Creating MIME Parts	46
8.2.1 Setting and Getting MIME Part Headers	46
8.2.2 Specifying an Optional Message Boundary Value	47
8.3 Writing MIME Messages	47
8.3.1 Example: WriteMIMEMessage()	47
8.4 Reading MIME Messages	48
9 Using FTP	49
9.1 Establishing an FTP Session	49
9.1.1 Translate Table for Commands	50
9.2 FTP File and System Methods	50
9.3 Using a Linked Stream to Upload Large Files	51
9.4 Customizing Callbacks Issued by the FTP Server	52
9.5 See Also	52

10 Using SSH	53
10.1 Creating an SSH Session	53
10.2 Supported Key Types and Hostkey Algorithms	54
10.3 Example: Listing Files via SFTP	54
10.4 Example: Authenticate with Keyboard Interactive	55
10.5 Example: Authenticate with Key Pair	55
10.6 Example: Execute a Remote Command	55
10.7 Example: Port Forward	56
10.8 See Also	56
11 Sending and Receiving IBM WebSphere MQ Messages	57
11.1 Overview	57
11.1.1 Getting Error Codes	58
11.2 Creating a Connection Object	58
11.2.1 Using the %Init() Method	58
11.2.2 Using the %Connect() Method	59
11.3 Specifying the Character Set (CCSID)	60
11.4 Specifying Other Message Options	60
11.5 Sending Messages	60
11.5.1 Example 1: SendString()	61
11.5.2 Example 2: SendCharacterStream()	61
11.5.3 Example 3: Sending a Message from the Terminal	62
11.6 Retrieving Messages	62
11.6.1 Example 1: ReceiveString()	62
11.6.2 Example 2: ReceiveCharacterStream()	63
11.7 Updating Message Information	63
11.8 Troubleshooting	64

List of Tables

Table 1–1: Example Properties for %Net.HttpRequest	4
--	---

1

Sending an HTTP Request

This page describes how to send HTTP requests (such as POST or GET).

1.1 Introduction to HTTP Requests

You create an instance of `%Net.HttpRequest` to send HTTP requests of various kinds and receive the responses. This object is equivalent to a web browser, and you can use it to make multiple requests. It automatically sends the correct cookies and sets the `Referer` header as appropriate.

To create an HTTP request, use the following general process:

1. Create an instance of `%Net.HttpRequest`.
2. Set properties of this instance to indicate the web server to communicate with. The basic properties are as follows:
 - Server specifies the IP address or machine name of the web server. The default is `localhost`.
Note: Do not include `http://` or `https://` as part of the value of `Server`. This causes `ERROR #6059: Unable to open TCP/IP socket to server http:/`.
 - Port specifies the port to connect to. The default is 80.
3. Optionally set other properties and call methods of your HTTP request, as described in [Specifying Other HTTP Request Properties](#).
4. Then send an HTTP request, by calling the **Get()** method or other methods of your instance of `%Net.HttpRequest`, as described in [Sending the HTTP Request](#).

You can make multiple requests from your instance, which will automatically handle cookies and the `Referer` header.

Note: If you created this HTTP request for use with the production outbound adapter (`EnsLib.HTTP.OutboundAdapter`), then instead use the methods of that adapter to send the request.

The HTTP response is then available in the `HttpResponse` property of your instance of `%Net.HttpRequest`; see [Using a HTTP Response](#).

5. Use the same instance of `%Net.HttpRequest` to send additional HTTP requests if needed. By default, InterSystems IRIS® data platform keeps the TCP/IP socket open so that you can reuse the socket without closing and reopening it.

For additional information, see [Managing Socket Reuse](#).

The following shows a simple example:

ObjectScript

```
set request=##class(%Net.HttpRequest).%New()  
set request.Server="tools.ietf.org"  
set request.Https=1  
set request.SSLConfiguration="TEST"  
set status=request.Get("/html/rfc7158")
```

For information on the `Https` and `SSLConfiguration` properties, see [Using SSL to Connect](#), later in this topic. Also, for a more complete version of this example, see [Accessing the HTTP Response](#).

1.2 HTTP Authentication

If the destination server requires login credentials, your HTTP request can include an `HTTP Authorization` header that provides the credentials. The following subsections provide the details:

- [Authenticating a Request When Using HTTP 1.0](#)
- [Authenticating a Request When Using HTTP 1.1](#)
- [Specifying the Authorization Header Directly](#)
- [Enabling Logging for HTTP Authentication](#)

If you are using a proxy server, you can also specify login credentials for the proxy server; see [Using a Proxy Server](#).

1.2.1 Authenticating a Request When Using HTTP 1.0

For HTTP 1.0, to authenticate an HTTP request, set the `Username` and `Password` properties of the instance of `%Net.HttpRequest`. The instance then creates the `HTTP Authorization` header based on that username and password, using the Basic Access Authentication ([RFC 2617](#)). Any subsequent request sent by this `%Net.HttpRequest` will include this header.

Important: Make sure to also use SSL (see [Using SSL to Connect](#)). In Basic authentication, the credentials are sent in base-64 encoded form and thus can be easily read.

1.2.2 Authenticating a Request When Using HTTP 1.1

For HTTP 1.1, to authenticate an HTTP request, in most cases, just set the `Username` and `Password` properties of the instance of `%Net.HttpRequest`. When an instance of `%Net.HttpRequest` receives a 401 HTTP status code and `WWW-Authenticate` header, it attempts to respond with an `Authorization` header that contains a supported authentication scheme. The first scheme that is supported and configured for InterSystems IRIS is used. By default, it considers these authentication schemes, in order:

1. Negotiate (SPNEGO and Kerberos, per [RFC 4559](#) and [RFC 4178](#))
2. NTLM (NT LAN Manager Authentication Protocol)
3. Basic (Basic Access Authentication as described in [RFC 2617](#))

Important: If there is a chance that Basic authentication will be used, make sure to also use SSL (see [Using SSL to Connect](#)). In Basic authentication, the credentials are sent in base-64 encoded form and thus can be easily read.

On Windows, if the Username property is not specified, InterSystems IRIS can instead use the current login context. Specifically, if the server responds with a 401 status code and a `WWW-Authenticate` header for SPNEGO, Kerberos, or NTLM, then InterSystems IRIS uses the current operating system username and password to create the `Authorization` header.

The details are different from the HTTP 1.0 case, as follows:

- If authentication succeeds, InterSystems IRIS updates the `CurrentAuthenticationScheme` property of the `%Net.HttpRequest` instance to indicate the authentication scheme that it used for the most recent authentication.
- If an attempt to get an authentication handle or token for a scheme fails, InterSystems IRIS saves the underlying error to the `AuthenticationErrors` property of the `%Net.HttpRequest` instance. The value of this property is a **\$LIST**, in which each item has the form *scheme* ERROR: *message*

Negotiate and NTLM are supported only for HTTP 1.1 because these schemes require multiple round trips, and HTTP 1.0 requires the connection to be closed after each request/response pair.

1.2.2.1 Variations

If you know the authentication scheme or schemes allowed by the server, you can bypass the initial round trip from the server by including an `Authorization` header that contains the initial token for the server for a chosen scheme. To do this, set the `InitiateAuthentication` property of the `%Net.HttpRequest` instance. For the value of this property, specify the name of a single authorization scheme allowed by the server. Use one of the following values (case-sensitive):

- Negotiate
- NTLM
- Basic

If you want to customize the authentication schemes to use (or change their order in which they are considered), set the `AuthenticationSchemes` of the `%Net.HttpRequest` instance. For the value of this property, specify a comma-separated list of authentication scheme names (using the exact values given in the previous list).

1.2.3 Specifying the Authorization Header Directly

For either HTTP 1.0 or HTTP 1.1 (if applicable to your scenario), you can specify the HTTP `Authorization` header directly. Specifically, you set the `Authorization` property equal to the authentication information required by the user agent for the resource that you are requesting.

If you specify the `Authorization` property, the `Username` and `Password` properties are ignored.

1.2.4 Enabling Logging for HTTP Authentication

To enable logging for the HTTP authentication, enter the following in the Terminal:

ObjectScript

```
set $namespace="%SYS"
kill ^ISCLOG
set ^%ISCLOG=2
set ^%ISCLOG("Category","HttpRequest")=5
```

Log entries are written to the `^ISCLOG` global. To write the log to a file (for easier readability), enter the following (still within the `%SYS` namespace):

ObjectScript

```
do ##class(%Auth2.Utils).DisplayLog("filename")
```

Where *filename* is the name of the file to create. The directory must already exist. If the file already exists, it is overwritten.

To stop logging, enter the following (still within the %SYS namespace):

ObjectScript

```
set ^%ISCLLOG=0
set ^%ISCLLOG("Category", "HttpRequest")=0
```

1.3 Specifying Other HTTP Request Properties

Before you send an HTTP request (see [Sending the HTTP Request](#)), you can specify its properties, as described in the following sections:

- [The Location Property](#)
- [Specifying the Internet Media Type and Character Encoding](#)
- [Using a Proxy Server](#)
- [Using SSL to Connect](#)
- [The HTTPVersion, Timeout, WriteTimeout, and FollowRedirect Properties](#)
- [Handling Chunked Responses](#)
- [Specifying Default Values for HTTP Requests](#)

You can specify default values for all properties of %Net.HttpRequest, as specified in the section listed last.

1.3.1 The Location Property

The Location property specifies the resource that you are requesting from the web server. If you set this property, then when you call the **Get()**, **Head()**, **Post()**, or **Put()** method, you can omit the location argument.

For example, suppose that you are sending an HTTP request to the URL `http://machine_name/test/index.html`

In this case, you would use the following values:

Table 1–1: Example Properties for %Net.HttpRequest

Properties	Value
Server	machine_name
Location	test/index.html

1.3.2 Specifying the Internet Media Type and Character Encoding

You can use the following properties to specify the [Internet media type](#) (also called *MIME type*) and character encoding in your instance of %Net.HttpRequest and its response:

- **ContentType** specifies the Content-Type header, which specifies the Internet media type of the request body. The default type is none.

Possible values include `application/json`, `application/pdf`, `application/postscript`, `image/jpeg`, `image/png`, `multipart/form-data`, `text/html`, `text/plain`, `text/xml`, and many others. For an extensive list, see <http://www.iana.org/assignments/media-types>.

- The `ContentCharset` property controls the desired character set for any content of the request if the content is of type `text` (`text/html` or `text/xml` for example). If you do not specify this property, InterSystems IRIS uses the default encoding of the InterSystems IRIS server.

Note: If you set this property, you must first set the `ContentType` property.

- The `NoDefaultContentCharset` property controls whether to include an explicit character set for content of type `text` if you have not set the `ContentCharset` property. By default, this property is false.

If this property is true, then if you have content of type `text` and if you have not set the `ContentCharset` property, no character set is included in the content type; this means that the character set `iso-8859-1` is used for the output of the message.

- The `WriteRawMode` property affects the entity body (if included). It controls how the body of the request is written. By default, this property is false and InterSystems IRIS writes the body in the encoding that is specified in the request headers. If this property is true, InterSystems IRIS writes the body in RAW mode (performing no translation of the character set).
- The `ReadRawMode` property controls how the body of the response is read. By default, this property is false and InterSystems IRIS assumes that the body is in the character set specified in the response headers. If this property is true, InterSystems IRIS reads the body in RAW mode (performing no translation of the character set).

1.3.3 Using a Proxy Server

You can send an HTTP request via a proxy server. In order to set this up, specify the following properties of your HTTP request:

- `ProxyServer` specifies the host name of the proxy server to use. If this property is not null, the HTTP request is directed to this machine.
- `ProxyPort` specifies the port to connect to, on the proxy server.
- `ProxyUsername` specifies the username to use when authenticating to the proxy server, if needed.
- `ProxyPassword` specifies the associated password.
- `ProxyHTTPS` controls whether the HTTP request is for an HTTPS page, rather than a normal HTTP page. This property is ignored if you have not specified a proxy server. This property changes the default port on the target system to 443, the proxy port. Also see [Using SSL to Connect](#).
- `ProxyTunnel` specifies whether to establish a tunnel through the proxy to the target HTTP server. If true, the request uses the HTTP CONNECT command to establish a tunnel. The address of the proxy server is taken from the `ProxyServer` and `ProxyPort` properties. If `ProxyHttps` is true, then once the tunnel is established, InterSystems IRIS negotiates the SSL connection. In this case, the `Https` property is ignored because the tunnel establishes a direct connection with the target system.

If `ProxyUsername` is set, and the proxy authorization scheme is Basic or Negotiate, the HTTP request object will set the `HTTP Proxy-Authorization` header based on the provided credentials. For other scenarios, set the `ProxyAuthorization` property of the HTTP request as needed.

For details, see the class documentation for `%Net.HttpRequest`.

1.3.4 Using SSL to Connect

The `%Net.HttpRequest` class supports SSL connections. To send the request via SSL, do the following:

1. Set the `SSLConfiguration` property to the name of the activated SSL/TLS configuration to use.

For information on creating and managing SSL/TLS configurations, see [InterSystems TCP Guide](#). The SSL/TLS configuration includes an option called **Configuration Name**, which is the string to use in this setting.

2. Also do one of the following, depending on whether you are using a proxy server:

- If you are not using a proxy server, set the `Https` property to true.
- If you are using a proxy server, set the `ProxyHTTPS` property to true.

In this case, to use an SSL connection to the proxy server itself, set the `Https` property to true.

Note that when you use an SSL connection to a given server, the default port on that server is assumed to be 443 (the HTTPS port). For example, if you are not using a proxy server and `Https` is true, this changes the default `Port` property to 443.

Also see [Using a Proxy Server](#).

1.3.4.1 Server Identity Checking

By default, when an instance of `%Net.HttpRequest` connects to a SSL/TLS secured web server, it checks whether the certificate server name matches the DNS name used to connect to the server. If these names do not match, the connection is not permitted. This default behavior prevents “man in the middle” attacks and is described in [RFC 2818](#), section 3.1; also see [RFC 2595](#), section 2.4.

To disable this check, set the `SSLCheckServerIdentity` property to 0.

1.3.5 The HTTPVersion, Timeout, WriteTimeout, and FollowRedirect Properties

`%Net.HttpRequest` also provides the following properties:

- `HTTPVersion` specifies the HTTP version to use when requesting a page. The default is "HTTP/1.1". You can also use the "HTTP/1.0".
- `Timeout` specifies how long, in seconds, to wait for a response from the web server. The default is 30 seconds.
- `WriteTimeout` specifies how long, in seconds, to wait for a write the web server to complete. By default it will wait indefinitely. The minimum accepted value is 2 seconds.
- `FollowRedirect` specifies whether to automatically follow redirection requests from the web server (signalled by the HTTP status codes in the range 300–399). The default is true if you are using GET or HEAD; otherwise it is false.

1.3.6 Handling Chunked Responses

If the server sends responses in chunks ([RFC9112](#)), you can provide custom handling for the responses as follows:

- Create a custom subclass of `%Net.ChunkedReader` and implement its **`ReadChunk()`** method.

This method receives a chunk as a string and should throw an exception in the case of an error. `%Net.HttpRequest` will catch this exception and return an error status.

- In your instance of `%Net.HttpRequest`, set the `ResponseChunkReader` property equal to an instance of your custom subclass.

If the `ResponseChunkReader` property is not null, the instance of `%Net.HttpRequest` sends the entire decoded response body in chunks through the applicable **`ReadChunk()`** method even if the response has gzip content-encoding.

1.3.7 Specifying Default Values for HTTP Requests

You can specify default values for all properties of %Net.HttpRequest.

- To specify a default value that apply to all namespaces, set the global node ^%SYS("HttpRequest", "propname") where "propname" is the name of the property.
- To specify a default value for one namespace, go to that namespace and set the node ^SYS("HttpRequest", "propname")
(The ^%SYS global affects the entire installation, and the ^SYS global affects the current namespace.)

For example to specify a default proxy server for all namespaces, set the global node ^%SYS("HttpRequest", "ProxyServer")

1.4 Setting and Getting HTTP Headers

You can set values for and get values of the HTTP headers.

Each of the following properties of %Net.HttpRequest contains the value of the HTTP header that has the corresponding name. These properties are automatically calculated if you do not set them:

- Authorization
- ContentEncoding
- ContentLength (This property is read-only.)
- ContentType (Specifies the [Internet media type](#) (MIME type) of the Content - Type header.)
- ContentCharset (Specifies the char set part of the Content - Type header. If you set this, you must *first* set the ContentType property.)
- Date
- From
- IfModifiedSince
- Pragma
- ProxyAuthorization
- Referer
- UserAgent

The %Net.HttpRequest class provides general methods that you can use to set and get the main HTTP headers. These methods ignore Content - Type and other entity headers.

ReturnHeaders()

```
method ReturnHeaders() as %String
```

Returns a string containing the main HTTP headers in this request.

OutputHeaders()

```
method OutputHeaders() as %Status
```

Writes the main HTTP headers to the current device.

GetHeader()

```
method GetHeader(name As %String) as %StringA
```

Returns the current value for any main HTTP header that has been set in this request. This method takes one argument, the name of the header (not case-sensitive); this is a string such as `Host` or `Date`

SetHeader()

```
method SetHeader(name As %String, value As %String) as %Status
```

Sets the value of a header. Typically you use this to set nonstandard headers; most of the usual headers are set via properties such as `Date`. This method takes two arguments:

1. The name of the header (not case-sensitive), without the colon (:) delimiter; this is a string such as `Host` or `Date`
2. The value of that header

You cannot use this method to set entity headers or read-only headers (`Content-Length` and `Connection`).

For details, see the class documentation for `%Net.HttpRequest`.

1.5 Managing Keep-Alive Behavior

If you reuse the same instance of `%Net.HttpRequest` to send multiple HTTP requests, by default, InterSystems IRIS keeps the TCP/IP socket open so that InterSystems IRIS does not need to close and reopen it.

If you do not want to reuse the TCP/IP socket, set the `SocketTimeout` property to 0.

The `SocketTimeout` property of `%Net.HttpRequest` specifies the window of time, in seconds, during which InterSystems IRIS will reuse a given socket. This timeout is intended to avoid using a socket that may have been silently closed by a firewall. The default value for this property is 115. You can set it to a different value.

1.6 Handling HTTP Request Parameters

When you send an HTTP request (see [Sending the HTTP Request](#)), you can include parameters in the *location* argument; for example: `"/test.html?PARAM=%25VALUE"` sets `PARAM` equal to `%VALUE`.

You can also use the following methods to control how your instance of `%Net.HttpRequest` handles parameters:

InsertParam()

```
method InsertParam(name As %String, value As %String)
```

Inserts a parameter into the request. This method takes two string arguments: the name of the parameter and the value of the parameter. For example:

ObjectScript

```
do req.InsertParam("arg1", "1")
```

You can insert more than one value for a given parameter. If you do so, the values receive subscripts starting with 1. Within other methods, you use these subscripts to refer to the intended value.

DeleteParam()

```
method DeleteParam(name As %String, index As %Integer = "") as %Boolean
```

Deletes a parameter from the request. The first argument is the name of the parameter. The second argument is the subscript for the value to delete; use this only if the request contains multiple values for the same parameter.

CountParam()

```
method CountParam(name As %String) as %Integer
```

Counts the number of values that are associated with a given parameter.

GetParam()

```
method GetParam(name As %String, default As %String = "", index As %Integer = 1) as %String
```

Gets the value of a given parameter in the request. The first argument is the name of the parameter. The second argument is the default value to return if the request does not have a parameter with this name; the initial value for this default is the null value. The third argument is the subscript for the value to get; use this only if the request contains multiple values for the same parameter.

IsParamDefined()

```
method IsParamDefined(name As %String, index As %Integer = 1) as %Boolean
```

Checks whether a given parameter is defined. This method returns true if the parameter has a value. The arguments are the same as for **DeleteParam()**.

NextParam()

```
method NextParam(name As %String) as %String
```

Retrieves the name of the next parameter, if any, after sorting the parameter names via **\$Order()**.

ReturnParams()

```
method ReturnParams() as %String
```

Returns the list of parameters in this request.

For details, see the class documentation for %Net.HttpRequest.

1.7 Including a Request Body

An HTTP request can include either a request body or form data. To include a request body, do the following:

1. Create an instance of %GlobalBinaryStream or a subclass. Use this instance for the EntityBody property of your HTTP request.

Note that %GlobalBinaryStream is deprecated but is still supported for use in this way. It is not recommended to substitute a different stream class for this use case.

2. Use the standard stream interface to write data into this stream. For example:

ObjectScript

```
Do oref.EntityBody.Write("Data into stream")
```

For example, you could read a file and use that as the entity body of your custom HTTP request:

ObjectScript

```
set file=##class(%File).%New("G:\customer\catalog.xml")
set status=file.Open("RS")
if $$$ISERR(status) do $System.Status.DisplayError(status)
set hr=##class(%Net.HttpRequest).%New()
do hr.EntityBody.CopyFrom(file)
do file.Close()
```

1.7.1 Sending a Chunked Request

If you use HTTP 1.1, you can send an HTTP request in chunks. This involves setting the Transfer-Encoding to indicate that the message is chunked, and using a zero-sized chunk to indicate completion.

Chunked encoding is useful when the server is returning a large amount of data and the total size of the response is not known until the request is fully processed. In such a case, you would normally need to buffer the entire message until the content length could be computed (which %Net.HttpRequest does automatically).

To send a chunked request, do the following:

1. Create a subclass of %Net.ChunkedWriter, which is an abstract stream class that defines an interface for writing data in chunks.

In this subclass, implement the **OutputStream()** method.
2. In your instance of %Net.HttpRequest, create an instance of your %Net.ChunkedWriter subclass and populate it with the request data that you want to send.
3. Set the EntityBody property of your %Net.HttpRequest instance equal to this instance of %Net.ChunkedWriter.

When you send the HTTP request (see [Sending the HTTP Request](#)), it calls the **OutputStream()** method of the EntityBody property.

In your subclass of %Net.ChunkedWriter, the **OutputStream()** method should examine the stream data, decide whether to chunk it and how to do so, and invoke the inherited methods of the class to write the output.

The following methods are available:

WriteSingleChunk()

```
method WriteSingleChunk(buffer As %String)
```

Accepts a string argument and writes the string as non-chunked output.

WriteFirstChunk()

```
method WriteFirstChunk(buffer As %String)
```

Accepts a string argument. Writes the appropriate Transfer-Encoding heading to indicate a chunked message, and then writes the string as the first chunk.

WriteChunk()

```
method WriteChunk(buffer As %String)
```

Accepts a string argument and writes the string as a chunk.

WriteLastChunk()

```
method WriteLastChunk(buffer As %String)
```

Accepts a string argument and writes the string as a chunk, followed by a zero length chunk to mark the end.

If non-null, the `TranslateTable` property specifies the [translation table](#) to use to translate each string as it is written. All of the preceding methods check this property.

1.8 Sending Form Data

An HTTP request can include either a request body or form data. To include a form data, use the following methods:

InsertFormData()

```
method InsertFormData(name As %String,
    value As %RawString,
    contentType As %String = "text/plain",
    filename As %String = "",
    writeRaw As %Boolean = 0)
```

Inserts form data into the request. This *name* argument is the name of the form item and *value* is the associated value. You can insert more than one value for a given form item. If you do so, the values receive subscripts starting with 1. Within other methods, you use these subscripts to refer to the intended value.

The arguments *contentType*, *filename*, and *writeRaw* are applicable for multipart forms. Each name-value pair can be represented with its own *contentType*. The optional *filename* argument can be used to set a filename in the Content-Disposition header for this part of a multipart form. If *writeRaw* is set to 1, then the value will be copied exactly as is in the multipart form. If set to 0, it will be converted using the character set associated with the *contentType* or with the `ContentCharset` property if the *contentType* does not specify a character set. When the value is a binary stream, use 1 for *writeRaw*.

DeleteFormData()

```
method DeleteFormData(name As %String, index As %Integer = "") as %Boolean
```

Deletes form data from the request. The first argument is the name of the form item. The second argument is the subscript for the value to delete; use this only if the request contains multiple values for the same form item.

CountFormData()

```
method CountFormData(name As %String) as %Integer
```

Counts the number of values associated with a given name, in the request.

IsFormDataDefined()

```
method IsFormDataDefined(name As %String, index As %Integer = 1) as %Boolean
```

Checks whether a given name is defined.

NextFormData()

```
method NextFormData(name As %String) as %String
```

Retrieves the name of the next form item, if any, after sorting the names via **\$Order()**.

1.8.1 Example 1

After inserting the form data, you generally call the **Post()** method. For example:

ObjectScript

```
Do httprequest.InsertFormData("element", "value")
Do httprequest.Post("/cgi-bin/script.CGI")
```

1.8.2 Example 2

For another example:

ObjectScript

```
Set httprequest=##class(%Net.HttpRequest).%New()
set httprequest.SSLConfiguration="MySSLConfiguration"
set httprequest.Https=1
set httprequest.Server="myserver.com"
set httprequest.Port=443
Do httprequest.InsertFormData("portalid", "2000000")
set tSc = httprequest.Post("/url-path/")
Quit httprequest.HttpResponse
```

1.9 Inserting, Listing, and Deleting Cookies

%Net.HttpRequest automatically manages cookies sent from the server; if the server sends a cookie, your instance of %Net.HttpRequest will return this cookie on the next request. (For this mechanism to work, you need to reuse the same instance of %Net.HttpRequest.)

Use the following methods to manage cookies within your instance of %Net.HttpRequest:

InsertCookie()

```
method InsertCookie(name As %String,
                    value As %String,
                    path As %String,
                    domain As %String,
                    expires As %String,
                    secure As %Boolean = 0) as %Status
```

Inserts a cookie into the request. Specify the following arguments:

- *name* is the name of the cookie.
- *value* is the value of the cookie.
- *path* is the path where the cookie should be stored.
- *domain* is the name of the machine from which to download the cookie.
- *expires* is the date and time when the cookie expires.
- *secure* is 1 if the cookie should be sent only over secure channels.

InsertCookieList()

Iterates through a multidimensional array that contains cookies and inserts each one into the request. The format of the array is as follows:

```
array(index)=$LB(name, domain, path, value, expires, secure)
```

GetFullCookieList()

```
method GetFullCookieList(ByRef cookies) as %Integer
```

Returns the number of cookies and returns (by reference) an array of cookies.

DeleteCookie()

```
method DeleteCookie(name As %String, path As %String, domain As %String) as %Status
```

Deletes a cookie from the request. The name, path, and domain must match the ones used to insert the cookie in the first place.

Remember that cookies are specific to an HTTP server. When you insert a cookie, you are using a connection to a specific server, and the cookie is not available on other servers.

1.10 Sending the HTTP Request

After you have created the HTTP request, use one of the following methods to send it:

Delete()**Class Member**

```
method Delete(location As %String = "",
              test As %Integer = 0,
              reset As %Boolean = 1) as %Status {}
```

Issues the HTTP DELETE request.

Get()**Class Member**

```
method Get(location As %String = "",
           test As %Integer = 0,
           reset As %Boolean = 1) as %Status {}
```

Issues the HTTP GET request. This method causes the web server to return the page requested.

Head()**Class Member**

```
method Head(location As %String,
            test As %Integer = 0,
            reset As %Boolean = 1) as %Status {}
```

Issues the HTTP HEAD request. This method causes the web server to return just the headers of the response and none of the body.

Patch()

Class Member

```
method Patch(location As %String = "",
             test As %Integer = 0,
             reset As %Boolean = 1) as %Status {}
```

Issues the HTTP PATCH request. Use this method to partial changes to an existing resource.

Post()

Class Member

```
method Post(location As %String = "",
            test As %Integer = 0,
            reset As %Boolean = 1) as %Status {}
```

Issues the HTTP POST request. Use this method to send data to the web server such as the results of a form, or upload a file. For an example, see [Sending Form Data](#).

Put()

Class Member

```
method Put(location As %String = "",
            test As %Integer = 0,
            reset As %Boolean = 1) as %Status {}
```

Issues the HTTP PUT request. Use this method to upload data to the web server. PUT requests are not common.

Send()

Class Member

```
method Send(type As %String,
            location As %String,
            test As %Integer = 0,
            reset As %Boolean = 1) as %Status {}
```

Sends the specified type of HTTP request to the server. This method is normally called by the other methods, but is provided for use if you want to use a different HTTP verb. Here *type* is a string that specifies an HTTP verb such as "POST".

In all cases:

- Each method returns a status, which you should check.
- If the method completes correctly, the response to this request will be in the *HttpResponse* property.
- The *location* argument is the URL to request, for example: `"/test.html"`.
- The *location* argument can contain parameters, which are assumed to be already URL-escaped, for example: `"/test.html?PARAM=%25VALUE"` sets PARAM equal to %VALUE.
- Use the *test* argument to check that you are sending what you are expecting to send:
 - If *test* is 1 then instead of connecting to a remote machine, the method will just output what it would have send to the web server to the current device.
 - If *test* is 2 then it will output the response to the current device after issuing the HTTP request.
- Each method automatically calls the **Reset()** method after reading the response from the server, except if *test*=1 or if *reset*=0.

The **Reset()** method resets the `%Net.HttpRequest` instance so that it can issue another request. This is much faster than closing this object and creating a new instance. This also moves the value of the `Location` header to the `Referer` header.

For example:

ObjectScript

```
Set httprequest=##class(%Net.HttpRequest).%New()  
Set httprequest.Server="www.intersystems.com"  
Do httprequest.Get("/")
```

For other examples, see the class documentation for `%Net.HttpRequest`.

1.11 Creating and Sending Multipart POST Requests

To create and send a multipart POST request, use the `%Net.MIMEPart` classes; see [Creating, Writing, and Reading MIME Messages](#).

1.12 Accessing the HTTP Response

After you send an HTTP request, the `HttpResponse` property of the request is updated. This property is an instance of `%Net.HttpResponse`. For details on working with this object, see [Using an HTTP Response](#).

1.13 See Also

- [Using an HTTP Response](#)
- `%Net.HttpRequest`
- [Creating, Writing, and Reading MIME Messages](#)

2

Using an HTTP Response

When you use an instance of `%Net.HttpRequest` to send an HTTP request, the response is available as the `HttpResponse` property of that instance. This property is an instance of `%Net.HttpResponse`.

This page describes how to use that instance.

2.1 Accessing the Data of the Response

The body of the HTTP response is contained in the `Data` property of the instance of `%Net.HttpResponse`. The `Data` property contains a stream object, specifically `%GlobalBinaryStream`. Note that `%GlobalBinaryStream` is deprecated but is still supported for use in this way. It is not recommended to substitute a different stream class for this use case.

To work with this stream, use the stream methods **`Write()`**, **`WriteLine()`**, **`Read()`**, **`ReadLine()`**, **`Rewind()`**, **`MoveToEnd()`**, and **`Clear()`**. You can also use the `Size` property of the stream.

The `ReadRawMode` property of the *request* controls how the body of the response is read.

- By default, this property is false and InterSystems IRIS assumes that the body is in the character set specified in the HTTP headers of the response (and translates the character set accordingly).
- If this property is true, InterSystems IRIS reads the body in RAW mode (performing no translation of the character set).

You can also use the **`OutputToDevice()`** method, which writes the full response to the current device. The headers are not in the same order as generated by the web server.

The following shows a simple example in which we copy the response stream to a file and save it:

ObjectScript

```
set request=##class(%Net.HttpRequest).%New()
set request.Server="tools.ietf.org"
set request.Https=1
set request.SSLConfiguration="TEST"
set status=request.Get("/html/rfc7158")
if $$$ISERR(status) {
    do $system.Status.DisplayError(status)
} else {
    set response=request.HttpResponse
}

Set file=##class(%FileCharacterStream).%New()
set file.Filename="c:/temp/rfc7158.html"
set status=file.CopyFrom(response.Data)
if $$$ISERR(status) {
    do $system.Status.DisplayError(status)
}
do file.%Close()
```

2.2 Accessing the HTTP Headers

The `%Net.HttpResponse` class stores its HTTP headers in an InterSystems IRIS multidimensional array. To access the headers, use the following methods:

GetHeader()

Returns the value of the given header.

GetNextHeader()

Returns the name of the next header after the given header.

Each of these methods takes a single argument, a string that is the name of an HTTP header.

You can also use the **OutputHeaders()** method, which writes the HTTP headers to the current device (although not in the same order they were generated).

2.3 Accessing Other Information about the Response

The `%Net.HttpResponse` class provides properties that store other specific parts of the HTTP response:

- `StatusLine` stores the HTTP status line, which is the first line of the response.
- `StatusCode` stores the HTTP status code.
- `ReasonPhrase` stores the human-readable reason that corresponds to `StatusCode`.
- `ContentInfo` stores additional information about the response body.
- `ContentType` stores the value of the `Content-Type`: header.
- `HttpVersion` indicates the version of HTTP that is supported by the web server that sent the response.

2.4 See Also

- [Sending an HTTP Request](#)
- `%Net.HttpRequest`

3

Email Support in InterSystems IRIS

This topic describes how InterSystems IRIS® data platform supports sending and receiving MIME email messages.

Note: InterSystems IRIS does not provide a mail server. Instead, it provides the ability to connect to and interact with mail servers.

3.1 Supported Email Protocols

Email sends messages across the Internet using standard protocols. InterSystems IRIS supports three of these protocols, as follows:

- InterSystems IRIS provides an object representation of MIME email messages. It supports text and non-text attachments, single-part or multipart message bodies, and headers in ASCII and non-ASCII character sets. (For more general support of MIME parts, see [Creating and Writing MIME Messages](#).)
- You can send email via an SMTP (Simple Mail Transport Protocol) server.
- You can also retrieve email from an email server via POP3, the most common standard for retrieving email from remote servers.

3.2 How InterSystems IRIS Represents MIME Email Messages

This section describes how InterSystems IRIS represents MIME email messages.

In general, a multipart MIME message consists of the following pieces:

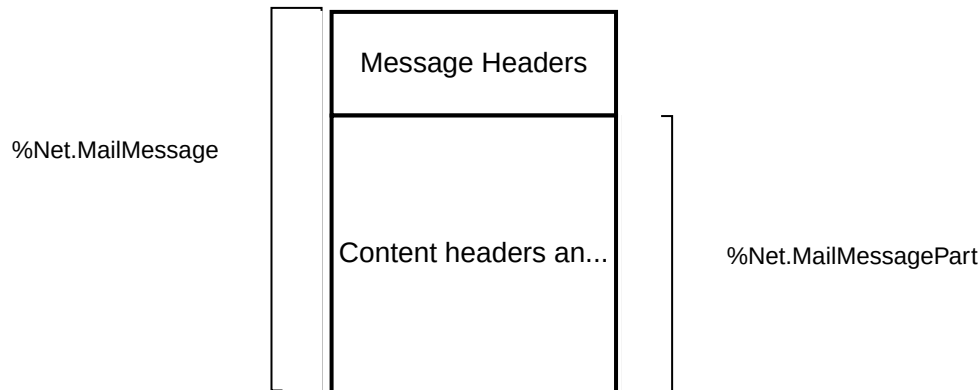
- A set of *message headers*, each of which contains information such as the address to which the message is sent. This also includes the `Mime-Type` header and the `Content-Type` header for the entire message.

For a multipart message, the `Content-Type` header must be `multipart/mixed` or some other subtype of `multipart`; the MIME standard has many variants.

- Multiple *message parts*, each of which consists of the pieces:
 - A set of *content headers*, which includes the `Content-Type` header and other headers specific to this part.

- A *body*, which is either text or binary, and which can be in a different character set than the bodies of other parts.

InterSystems IRIS uses two classes to represent email messages: `%Net.MailMessage` and `%Net.MailMessagePart`, the superclass of `%Net.MailMessage`. The following graphic shows the relationship between these classes:



In general:

- To represent an ordinary, one-part message, you use `%Net.MailMessage`
- To represent a multipart message, you use `%Net.MailMessage` as the parent message and you use multiple instances of `%Net.MailMessagePart` as its parts.

3.3 Email Support in Interoperability Productions

Within an interoperability production, you can include a custom business service that periodically checks a mailbox, finds matches, sends the messages (as instances of `%Net.MailMessage`) to the associated business service, and deletes the messages from the email server. The business service, which you create and configure, uses these messages and communicates with the rest of the production. See [Using the Email Inbound Adapter](#).

Similarly, you can include a custom business operation that sends email. See [Using the Email Outbound Adapter](#).

3.4 See Also

- [Creating Email Messages](#)
- [Sending Email Messages](#)
- [Retrieving Email Messages](#)
- [Working with Received Email](#)
- [Creating and Writing MIME Messages](#)

4

Creating Email Messages

This page describes how to create email messages in InterSystems IRIS data platform®.

Note: You should be aware of the requirements of the SMTP server that you are using. For example, some SMTP servers require that you include a Subject header. Similarly, some SMTP servers do not permit arbitrary From headers. Similarly, some SMTP servers recognize the Priority header and others recognize X-Priority instead.

4.1 Creating Single-Part Email Messages

To create a single-part email message, you use the `%Net.MailMessage` class. To create a mail message, do the following:

1. Create an instance of `%Net.MailMessage`.

Tip: You can specify a character set as the argument to `%New()`; if you do so, that sets the `Charset` property for the message. For information on how this affects the message, see [Automatic Encoding and Character Translation](#).

2. Set the `To`, `From`, and `Subject` properties of your instance.
 - `To` — The list of email addresses to which this message will be sent. This property is a standard InterSystems IRIS list class; to work with it, you use the standard list methods: **`Insert()`**, **`GetAt()`**, **`RemoveAt()`**, **`Count()`**, and **`Clear()`**.
 - `From` — The email address this message is sent from.
 - `Subject` — The subject of the message, if this is required by the SMTP server you are using.
3. Optionally set `Date`, `Cc`, `Bcc`, and other properties. For details, see [Specifying Basic Email Headers](#).
4. If the message is not plain text, set the following properties to indicate the kind of message you are creating:
 - If this is an HTML message, set the `IsHTML` property to 1.
 - If this is a binary message, set the `IsBinary` property to 1.
5. To specify the character set of the message and its headers, set the `Charset` property as needed. (For details on how this affects the message, see [Automatic Encoding and Character Translation](#).)

Important: It is important to specify the character set *before* you add the contents of the message.

6. Add the contents of the message:

- For plain text or HTML, use the `TextData` property, which is an instance of `%FileCharacterStream`. You do not need to specify the `TranslateTable` property of this stream; that occurred automatically when you specified the character set of the mail message.
- For binary data, use the `BinaryData` property, which is an instance of `%FileBinaryStream`.

Note that `%FileCharacterStream` and `%FileBinaryStream` are deprecated but are still supported for use in this way. It is not recommended to substitute different stream classes for this use case.

Tip: When you specify the `Filename` property of the stream, be sure to use a directory to which the users will have write access.

To work with these properties, use the standard stream methods: **`Write()`**, **`WriteLine()`**, **`Read()`**, **`ReadLine()`**, **`Rewind()`**, **`MoveToEnd()`**, and **`Clear()`**. You can also use the `Size` property of the stream, which gives you the size of the message contents.

The following method creates a simple message and specifies the addresses for it:

Class Member

```
ClassMethod CreateTextMessage() As %Net.MailMessage
{
    Set msg = ##class(%Net.MailMessage).%New()
    Set msg.From = "test@test.com"
    Do msg.To.Insert("xxx@xxx.com")
    Do msg.Cc.Insert("yyy@yyy.com")
    Do msg.Bcc.Insert("zzz@zzz.com")
    Set msg.Subject="subject line here"
    Set msg.IsBinary=0
    Set msg.IsHTML=0
    Do msg.TextData.Write("This is the message.")

    Quit msg
}
```

4.2 Creating Multipart Email Messages

To create a multipart email message:

1. Create an instance of `%Net.MailMessage`.

Tip: You can specify a character set as the argument to `%New()`; if you do so, that sets the `Charset` property for the message. For information on how this affects the message, see [Automatic Encoding and Character Translation](#).

2. Set the `To`, `From`, and `Subject` properties of this instance. Optionally set other properties to specify other message headers.
3. Set the `IsMultiPart` property to 1.
4. Set the `MultiPartType` property to one of the following: "related", "alternative", or "mixed". This affects the `Content - Type` header of the entire message.
5. For each part that the message should contain, create an instance of `%Net.MailMessagePart` and specify its properties as described in [Creating Single-part Email Messages](#) — starting with step 4.
6. For the parent email message, set the `Parts` property, which is an array. Insert each child message part into this array.

When you send the message, the `%Net.SMTP` class automatically sets the `Content - Type` header for the message as appropriate (given the value of the `MultiPartType` property).

4.3 Specifying Email Message Headers

As noted previously, both the message itself and each part of a message has a set of headers.

The `%Net.MailMessage` and `%Net.MailMessagePart` classes provide properties that give you easy access to the most commonly used headers, but you can add any header you need. This section provides information on all the headers as well as how to create custom headers.

The headers of a given message part are in the character set specified by the `Charset` property of that part.

Note: You should be aware of the requirements of the SMTP server that you are using. For example, some SMTP servers require that you include a `Subject` header. Similarly, some SMTP servers do not permit arbitrary `From` headers.

Similarly, some SMTP servers recognize the `Priority` header and others recognize `X-Priority` instead.

4.3.1 Specifying Basic Email Headers

Set the following properties (only in `%Net.MailMessage`) to set the most commonly used headers of the message itself:

- **To** — (Required) The list of email addresses to which this message will be sent. This property is a standard InterSystems IRIS list; to work with it, you use the standard list methods: **Insert()**, **GetAt()**, **RemoveAt()**, **Count()**, and **Clear()**.
- **From** — (Required) The email address this message is sent from.
- **Date** — The date of this message.
- **Subject** — (Required) A string containing the subject for this message.
- **Sender** — The actual sender of the message.
- **Cc** — The list of carbon copy addresses to which this message will be sent.
- **Bcc** — The list of blind carbon copy addresses to which this message will be sent.

4.3.2 Content-Type Header

When you send the message, the `Content - Type` header for the message and for each message part is automatically set as follows:

- If the message is plain text (`IsHTML` equals 0 and `IsBinary` equals 0), the `Content - Type` header is set to `"text/plain"`.
- If the message is HTML (`IsHTML` equals 1 and `IsBinary` equals 0), the `Content - Type` header is set to `"text/html"`.
- If the message is binary (`IsBinary` equals 1), the `Content - Type` header is set to `"application/octet-stream"`.
- If the message is multipart, the `Content - Type` header is set as appropriate for the value of the `MultiPartType` property.

Both `%Net.MailMessage` and `%Net.MailMessagePart` provide the `ContentType` property, which gives you access to the `Content - Type` header.

4.3.3 Content-Transfer-Encoding Header

Both `%Net.MailMessage` and `%Net.MailMessagePart` provide the `ContentTransferEncoding` property, which provides an easy way to specify the `Content - Transfer - Encoding` header of the message or the message part.

This property can be one of the following: `"base64"` `"quoted-printable"` `"7bit"` `"8bit"`

The default is as follows:

- For a binary message or message part: "base64"

Important: Note that if the content is "base64" encoded, it cannot contain any Unicode characters. If the content you wish to send includes Unicode characters, then make sure to use [\\$ZCONVERT](#) to convert the content to UTF-8, and then base-64 encode it. For example:

ObjectScript

```
set BinaryText=$ZCONVERT(UnicodeText,"0","UTF8")
set Base64Encoded=$system.Encryption.Base64Encode(BinaryText)
```

The recipient must use the reverse process to decode the text:

ObjectScript

```
set BinaryText=$system.Encryption.Base64Decode(Base64Encoded)
set UnicodeText=$ZCONVERT(BinaryText,"1","UTF8")
```

- For a text message or message part: "quoted-printable"

Also see [Automatic Encoding and Character Translation](#).

4.3.4 Custom Headers

With both %Net.MailMessage and %Net.MailMessagePart, you can set or get custom headers by accessing the Headers property, which is an array with the following structure:

Array Key	Array Value
Name of the header, such as "Priority"	Value of the header

You use this property to contain additional headers such as X-Priority and others. For example:

ObjectScript

```
do msg.Headers.SetAt(1,"X-Priority")
do msg.Headers.SetAt("High","X-MSMail-Priority")
do msg.Headers.SetAt("High","Importance")
```

Different email servers and clients recognize different headers, so it can be useful to set multiple similar headers to be sure that the server or client receives a message with a header it can recognize.

4.4 Adding Attachments to a Message

You can add attachments to an email message or message part (specifically, to an instance of %Net.MailMessagePart or %Net.MailMessage). To do so, use the following methods:

Each of these methods adds the attachment to the Parts array of the original message (or message part), and automatically sets the IsMultiPart property to 1.

AttachFile()

Class Member

```
method AttachFile(Dir As %String,
                  File As %String,
                  isBinary As %Boolean = 1,
                  charset As %String = "",
                  ByRef count As %Integer) as %Status {}
```

Attaches the given file to the email message. By default the file is sent as a binary attachment, but you can specify instead that it is text. You can also specify the character set that the file uses if it is text.

Specifically, this method creates an instance of %Net.MailMessagePart and places the contents of the file in the BinaryData or TextData property as appropriate, and sets the Charset property and TextData.TranslateTable properties if needed. The method returns, by reference, an integer that indicates the position of this new message part within the Parts array.

This method also sets the Dir and FileName properties of the message or message part.

AttachStream()

Class Member

```
method AttachStream(stream As %Stream.Object,
                    Filename As %String,
                    isBinary As %Boolean = 1,
                    charset As %String = "",
                    ByRef count As %Integer) as %Status {}
```

Attaches the given stream to the email message. The attachment is considered a file attachment if *Filename* is specified. Otherwise it is considered an inline attachment. See the comments for **AttachFile()**.

AttachNewMessage()

Class Member

```
method AttachNewMessage() as %Net.MailMessagePart {}
```

Creates a new instance of %Net.MailMessage, adds it to the message, and returns the newly modified parent message or message part.

AttachEmail()

Class Member

```
method AttachEmail(mailmsg As %Net.MailMessage) {}
```

Given an email message (an instance of %Net.MailMessage), this method adds it to the message. This method also sets the Dir and FileName properties of the message or message part.

Note: This method sets ContentType to "message/rfc822". In this case, you cannot add any other attachments.

4.4.1 Example: MessageWithAttachment()

The following example generates a simple email message with one hardcoded attachment.

Class Member

```
ClassMethod MessageWithAttachment() As %Net.MailMessage
{
  Set msg = ##class(%Net.MailMessage).%New()
  Set msg.Subject="Message with attachment "_$h
  Set msg.IsBinary=0
  Set msg.IsHTML=0
  Do msg.TextData.Write("This is the main message body.")

  //add an attachment
  Set status=msg.AttachFile("c:\", "GNET.pdf")
  If $$$ISERR(status) {
    Do $System.Status.DisplayError(status)
    Quit $$$NULLOREF
  }
  Quit msg
}
```

For other examples, see the class reference for the %Net.MailMessagePart class.

4.5 Encoding and Character Translation

A email message part contains information about both the character sets used and the content-transfer-encoding used (if any). For reference, this section describes how this information is used.

For information on character sets and translation tables, see [Translation Tables](#).

%Net.SMTP checks the Charset property of each part and then applies the appropriate [translation table](#).

If you do not specify the Charset property for a given part, InterSystems IRIS uses UTF-8.

%Net.SMTP also checks the ContentTransferEncoding property. If this property is "base64" or "quoted-printable", then when it creates the message, %Net.SMTP encodes the body as needed. (If the content transfer encoding is "7bit" or "8bit", no encoding is needed.)

Important: Note that if the content is "base64" encoded, it cannot contain any Unicode characters. If the content you wish to send includes Unicode characters, then make sure to use \$ZCONVERT to convert the content to UTF-8.

4.6 See Also

- [Email Support in InterSystems IRIS](#)
- [Sending Email Messages](#)
- [Creating and Writing MIME Messages](#)

5

Sending Email via SMTP

This page describes how to use %Net.SMTP to send MIME email messages.

5.1 Overview

If you have access to an SMTP server, you can send email messages, as follows:

1. Create an instance of %Net.SMTP and [authenticate to the SMTP server](#).
2. Create the email message to send (as described in [Creating Email Messages](#)).
3. [Send the message](#) and check the status returned by the SMTP instance.

5.2 Authenticating to the SMTP Server

To authenticate to the SMTP server, do the following:

1. Create an instance of %Net.SMTP and set its properties as needed, especially the following:
 - smtpserver is the name of the SMTP server you are using.
 - port is the port you are using on the SMTP server; the default is 25.
 - timezone specifies the time zone of the server, as specified by [RFC 822](#), for example "EST" or "-0400" or "LOCAL". If this is not set, the message uses universal time.

This object describes the SMTP server you will use. For example:

```
Set server=##class(%Net.SMTP).%New()  
Set server.smtpserver="smtp.mysmtpserver.com"  
Set server.port=25
```

2. Specify the necessary credentials to authenticate to the server. To do so:
 - a. Create an instance of %Net.Authenticator.
 - b. Set the UserName and Password properties of this object.
 - c. Set the authenticator property of your %Net.SMTP instance equal to this object.
 - d. If the message itself has an authorized sender, set the AuthFrom property of your %Net.SMTP instance.

- e. Set MechanismList, which specifies what authentication methods should be attempted. The mail server retains a list of accepted authentication mechanisms and the first mechanism in the MechanismList that is accepted will be used to connect to the server. Define a MechanismList that suits your application requirements and the server setting.

For example:

```
//Create object to carry authentication
Set auth=##class(%Net.Authenticator).%New()
Set auth.UserName="iscstest@mysmtpserver.com"
Set auth.Password="123pass" //should instead get password from a password store

Set server.authenticator=auth
Set server.AuthFrom=auth.UserName
```

For an alternative, see [Enabling XOAUTH2 Authentication](#).

3. To use an SSL/TLS connection to the SMTP server:

- a. Set the SSLConfiguration property to the name of the activated SSL/TLS configuration to use.

For information on creating and managing SSL/TLS configurations, see [InterSystems TLS Guide](#). The SSL/TLS configuration includes an option called **Configuration Name**, which is the string to use in this setting.

- b. Set the UseSTARTTLS property to either 0 or 1.

In most cases, use the value 0. Use the value 1 for the case in which the server interaction begins on a normal TCP socket and then switches to TLS on the same port as the normal socket. For details, see [RFC 3207](#).

- c. Optionally set the SSLCheckServerIdentity property to 1. Do this if you want to verify the host server name in the certificate.

5.2.1 Enabling XOAUTH2 Authentication

%Net.SMTP supports XOAUTH2 as an authentication method.

%Net.Authenticator has an access token property, which allows XOAUTH2 to be included in its mechanism list. If XOAUTH2 is included, %Net.Authenticator will use it as an authentication method; it will not do so otherwise. Users who do not wish to use XOAUTH2 with %Net.SMTP do not need to do anything.

5.3 Sending Email

Once you have an instance of %Net.SMTP that is authenticated to an SMTP server, send email as follows:

1. Create the email message to send (as described in [Creating Email Messages](#)).
2. Call the **Send()** method of your SMTP instance. This method returns a status, which you should check.
3. If the returned status indicates an error, check the Error property, which contains the error message itself.

By default, %Net.SMTP warns about authentication errors, but if you set the IgnoreFailedAuth property to 1, it will not do so. If you set the IgnoreFailedAuth property to 1, then you also need to check for authentication errors; to do so, check the value of the AuthWarning property. If IgnoreFailedAuth is 1, the AuthWarning property is updated with every call to the **Send()** method, to indicate if there was an authentication error. If IgnoreFailedAuth is 1, the AuthWarning property is not updated and should not be used.

4. Check the FailedSend property, which contains a list of email addresses for which the send action failed.

Important: %Net.SMTP writes the message body into a temporary file stream. By default, this file is written to the namespace directory and if the directory requires special write permissions, the file is not created and you get an empty message body.

You can define a new path for these temporary files and choose a path that does not restrict write access (for example, /tmp). To do so, set the global node %SYS("StreamLocation", namespace) where namespace is the namespace in which your code is running. For example:

ObjectScript

```
Set ^%SYS("StreamLocation", "SAMPLES")="/tmp"
```

If %SYS("StreamLocation", namespace) is null, then InterSystems IRIS uses the directory specified by %SYS("TempDir", namespace). If %SYS("TempDir", namespace) is not set, then InterSystems IRIS uses the directory specified by %SYS("TempDir")

5.4 Other Properties of %Net.SMTP

The %Net.SMTP class also has some other properties that you might need, depending on the SMTP server you are using:

- AllowHeaderEncoding specifies whether the **Send()** method encodes non-ASCII header text. The default is 1, which means that non-ASCII header text is encoded as specified by [RFC 2047](#).
- ContinueAfterBadSend specifies whether to continue trying to send a message after detecting a failed email address. If ContinueAfterBadSend is 1, the system will add the failed email address to the list in the FailedSend property. The default is 0.
- ShowBcc specifies whether the Bcc headers are written to the email message. These will normally be filtered out by the SMTP server.

5.5 See Also

- [Email Support in InterSystems IRIS](#)
- [Using the Email Outbound Adapter \(for interoperability productions\)](#)
- [Creating Email Messages](#)

6

Retrieving Email via POP3

This page discusses how to use the %Net.POP3 class to retrieve email messages.

6.1 Overview

If you have the needed permissions and if the mail server is running, you can download and process email messages from it using the POP3 protocol, as follows:

1. Create an instance of %Net.POP3 and [connect to the POP3 server](#).
2. [Examine the mailbox](#).
3. Optionally, to prevent the connection from timing out, call the **Ping()** method of your %Net.POP3 instance.
4. [Retrieve messages](#).
5. [Update the mailbox](#).

6.2 Connecting to the Server

To connect to a POP3 server:

1. Create an instance of %Net.POP3. This object describes the mail server you will use.
2. Optionally specify the following properties of your instance of %Net.POP3:
 - port — Specifies the port you will use; the default is 110.
 - timeout — Specifies the read timeout in seconds; the default is 30 seconds.
 - StoreAttachToFile — Specifies whether to save each attachment to a file, when a message is read (when the message includes the content-disposition; attachment header). The default is false.

Note that this setting does nothing unless AttachDir is also set.

- StoreInlineToFile — Specifies whether to save each inline attachment to a file, when a message is read (when the message includes the content-disposition; inline header). The default is false.

Note that this setting does nothing unless AttachDir is also set.

- **AttachDir** — Specifies the directory into which the attachment are saved. There is no default. Make sure to terminate the name of the directory with a slash (/) or backslash (\), as appropriate for the operating system. Also make sure that this is directory already exists, and the users have write access to it.
- **IgnoreInvalidBase64Chars** — Specifies whether to ignore invalid characters found during base-64 decoding. The default is false (and invalid characters result in an error). Note that [RFC 2045](#) is ambiguous about whether unexpected characters should be ignored or should result in an error during base-64 decoding.

3. To use an SSL/TLS connection to the server:

- a. Set the **SSLConfiguration** property to the name of the activated SSL/TLS configuration to use.

For information on creating and managing SSL/TLS configurations, see [InterSystems TLS Guide](#). The SSL/TLS configuration includes an option called **Configuration Name**, which is the string to use in this setting.

- b. Set the **UseSTARTTLS** property to either 0 or 1.

In most cases, use the value 0. Use the value 1 for the case in which the server interaction begins on a normal TCP socket and then switches to TLS on the same port as the normal socket. For details, see [RFC 2595](#).

Note that if **UseSTARTTLS** is set to 1, connect to the server over port 110, instead of the default 995.

- c. Optionally set the **SSLCheckServerIdentity** property to 1. Do this if you want to verify the host server name in the certificate.

4. Call the **Connect()** method of your instance. This method takes three arguments, in order:

- a. The name of the POP3 server
- b. A username
- c. A password

For an alternative, see [Enabling XOAUTH2 Authentication](#).

Each of these methods returns a status, which you should check before continuing. Also see the class reference for `%Net.POP3` for complete method signatures.

For example:

```
Set server=##class(%Net.POP3).%New()  
  
//Our POP3 server uses the default port  
//but let's set it anyway  
Set server.port=110  
  
//just in case we plan to fetch any messages  
//that have attachments  
Set server.StoreAttachToFile=1  
Set server.StoreInlineToFile=1  
Set server.AttachDir="c:\DOWNLOADS\  
  
Set servername="pop.mypop3server.com"  
Set user="isctest@mypop3server.com"  
Set pass="123pass"  
  
Set status=server.Connect(servername,user,pass)  
If $$$ISERR(status) {  
    Do $System.Status.DisplayError(status)  
    Quit $$$NULLOREF  
}
```

6.2.1 Enabling XOAUTH2 Authentication

`%Net.POP3` supports XOAUTH2 as an authentication method.

To use XOAUTH2 with %Net.POP3, pass an access token as a parameter to **%Net.POP3.Connect()**. If passed an access token, this method will attempt to use XOAUTH2 regardless of whether or not a password is supplied. Users who do not wish to use XOAUTH2 should be careful not to pass in an access token parameter.

6.3 Getting Information about the Mailbox

While you are connected to a POP3 server, you are logged in to a user account and have access to the mailbox for that user account. Use the following methods to find what the mailbox contains:

GetMailBoxStatus()

Returns, by reference, the number of messages in the mailbox and the number of bytes that the mailbox uses.

GetMessageUIDArray()

If given an empty string as the first argument, this method returns, by reference, an array of information about the messages in the mailbox (excluding any that are currently marked for deletion). Each element in this array contains the following information about one message:

Array Key	Array Item
Number of the message, within the mailbox in its current state. The first message is number 1, and so on. The message number of a given message is not guaranteed to be the same in all sessions.	Unique message identifier (UID), which is the permanent identifier of this message available in all sessions. UIDs are unique to each mailbox.

GetSizeOfMessages()

If given an empty string as the first argument, this method returns, by reference, an array of information about the messages in the mailbox (excluding any that are currently marked for deletion). Each element in this array contains the following information about one message:

Array Key	Array Item
Number of the message, within the mailbox in its current state.	Size of this message, in bytes.

Each of these methods returns a status, which you should check before continuing. Also see [Summary of Methods](#).

For example, the following method writes information about the mailbox that we are currently accessing:

Class Member

```
ClassMethod ShowMailbox(server as %Net.POP3)
{
    Set status=server.GetMailBoxStatus(.count,.size)
    If $$$ISERR(status) {
        Do $System.Status.DisplayError(status)
        Quit
    }
    Write "Mailbox information *****",!
    Write "Number of messages in mailbox: ",count,!
    Write "Size of messages: ",size,!

    Set status=server.GetMessageUIDArray(.uids)
    Set status=server.GetSizeOfMessages(.sizes)
```

```
//iterate through messages, get info, and write it
For i=1:1:count {
    Set uid=uids.GetAt(i)
    Set size=sizes.GetAt(i)
    Write "Msg number:", i, "    UID:",uid, "    size:",size,!
}
}
```

This method generates output similar to the following:

```
Mailbox information *****
Number of messages in mailbox: 4
Size of messages: 18634
Msg number:1    UID:6ef78df6fd660391    size:7245
Msg number:2    UID:7410041a6faf4a87    size:5409
Msg number:3    UID:5555af7fa489e406    size:5121
Msg number:4    UID:299ad2b54c01a6be    size:859
```

6.4 Retrieving Messages

Once you have connected to the server and have examined the mailbox, retrieve messages with the following messages of your instance of %Net.POP3:

Fetch()

Given a message number as the first argument, this method returns (by reference, as the second argument) an instance of %Net.MailMessage that contains that message.

FetchMessage()

Given a message number as the first argument, this method returns (by reference) information such as the From and To and other common headers, an array containing *all* the headers (including the common ones), and the message contents themselves

Each of these methods returns a status, which you should check before continuing. Note that these methods return an error status if the message is currently marked for deletion.

Also see [Summary of Methods](#), which shows the complete method signatures for **Fetch()** and **FetchMessage()**.

6.4.1 Encoding and Character Translation of Messages

A email message part contains information about both the character sets used and the content-transfer-encoding used (if any). For reference, this section describes how this information is used.

%Net.POP3 checks the Content - Transfer - Encoding header of each message part and decodes the body as needed.

Then %Net.POP3 checks the Content - Type header of each message part. This affects the Charset property of the message part and also controls the [translation table](#) used when the message part is created in InterSystems IRIS.

For information on character sets and translation tables, see [Translation Tables](#).

6.5 Retrieving Attachments

The Content - Disposition header might specify attachment, with or without a filename. For example:

```
Content-Disposition: attachment; filename=genome.jpeg;
```

If the `Content-Disposition` header does specify attachment, your `%Net.POP3` instance can save all attachments in the message to files. To make this happen:

1. Specify the following properties of your `%Net.POP3` instance:
 - Specify `StoreAttachToFile` as 1.
 - Specify `StoreInlineToFile` as 1.
 - Specify a valid directory for `AttachDir`. Make sure to terminate the name of the directory with a slash (/) or backslash (\), as appropriate for the operating system. Also make sure that this is directory already exists, and the users have write access to it.
2. Call **`Fetch()`** or **`FetchMessage()`** of your `%Net.POP3` instance.

Each filename is determined as follows:

1. If the `Content-Disposition` header specifies a filename, that filename is used.
2. Otherwise, if the `Content-Type` header specifies a filename, that filename is used.
3. Otherwise, the system creates a name of the form `ATTxxxxxx.dat`.

Note the following points:

- If the file already exists, the attachment is not downloaded.
- There is no default for `AttachDir`.
- The size of the attachment is not limited by InterSystems IRIS but might be limited by the file system.
- The `Dir` and `FileName` properties are not used here. They are relevant only when you upload an attachment to a mail message, as described in [Adding Attachments to a Message](#).

6.6 Retrieving Attached Messages

While you are connected to a mailbox, you can download any email messages that are attached to the email messages in the inbox. To do so, use the **`GetAttachedEmail()`** method of your `%Net.POP3` instance to retrieve the contents of the enclosed email.

Given an instance of `%Net.MailMessagePart`, this method returns a single-part message that has contents of that message part. Specifically, it returns (as an output parameter) an instance of `%Net.MailMessage` initialized with the data taken from the attached email message.

6.7 Deleting Messages

While you are connected to a mailbox, you can mark messages for deletion in that mailbox. You can do this in a couple of ways.

- You can use the **`DeleteMessage()`** method. This method takes one argument, the message number to delete.
- When you retrieve a message with the **`Fetch()`** or **`FetchMessage()`** method, you can specify an optional argument that tells the POP3 server to mark the message for deletion after you have retrieved it.

Remember the following points:

- These methods do not delete a message; they mark it for deletion. The message is not deleted until you complete the POP3 transaction with **QuitAndCommit()**. If you simply disconnect from the server, your changes are discarded.
- You can call the **RollbackDeletes()** method to change the messages so that they are no longer marked for deletion.
- Each of these methods returns a status, which you should check.

6.8 Updating the Mailbox

After examining the mailbox and retrieving messages, you may want to update the mailbox:

1. Optionally, if you have marked messages for [deletion](#) but now choose not to delete them, call the **RollbackDeletes()** method of your %Net.POP3 instance.
2. When you are done making changes to the mailbox, call one of the following methods:
 - **QuitAndCommit()** — Commits your changes and logs out of the mail server.
 - **QuitAndRollback()** — Rolls back your changes and logs out of the mail server.

6.9 Summary of Methods

This section lists all the methods of %Net.POP3 that you can use to examine and retrieve messages.

Fetch()

Class Member

```
method Fetch(MessageNumber As %Integer,  
             ByRef MailMsg As %Net.MailMessage,  
             Delete As %Boolean = 0,  
             messageStream As %BinaryStream) as %Status {}
```

Returns (by reference) the message indicated by *MessageNumber* and optionally marks the message for deletion. Note that this method returns an error status if the message is already marked for deletion.

If *messageStream* is specified, then the original message is written to this binary stream.

FetchFromStream()

Class Member

```
method FetchFromStream(messageStream As %BinaryStream, ByRef Msg As %Net.MailMessage) as %Status  
{} 
```

This method is for use when you specify the *messageStream* argument for **Fetch()**.

Retrieves a single email message from the given binary stream. *messageStream* must be a binary stream containing the message. The message is returned by reference in *Msg*. This could be a multipart message.

FetchMessage()**Class Member**

```
method FetchMessage(MessageNumber As %Integer,
    ByRef From As %String,
    ByRef To As %String,
    ByRef Date As %String,
    ByRef Subject As %String,
    ByRef MessageSize As %Integer,
    ByRef MsgHeaders As %ArrayOfDataTypes,
    ByRef MailMsg As %Net.MailMessage,
    Delete As %Boolean = 0) as %Status {}
```

Returns (by reference) specific message headers, the message size, the message header array, and the message itself and optionally marks the message for deletion. Note that this method returns an error status if the message is already marked for deletion.

FetchMessageHeaders()**Class Member**

```
method FetchMessageHeaders(MessageNumber As %Integer,
    ByRef MsgHeadersArray As %String) as %Status {}
```

Given a message number, this method returns (by reference) an array containing all the headers of that message. This method returns an error status if the message is currently marked for deletion.

FetchMessageInfo()**Class Member**

```
method FetchMessageInfo(MessageNumber As %Integer,
    Lines As %Integer,
    ByRef From As %String,
    ByRef To As %String,
    ByRef Date As %String,
    ByRef Subject As %String,
    ByRef MessageSize As %Integer,
    ByRef MsgHeaders As %ArrayOfDataTypes,
    ByRef MessageText As %String) as %Status {}
```

Given a message number, this method returns (by reference) specific message headers, the message size, the message header array, and the given number of lines of text from this message. This method returns an error status if the message is currently marked for deletion.

GetAttachedEmail()**Class Member**

```
method GetAttachedEmail(msgpart As %Net.MailMessagePart,
    Output mailmsg As %Net.MailMessage) as %Status {}
```

Given a message part, this method returns (as an output parameter) a single-part email message that is initialized with the data from the message part.

GetMessageUID()**Class Member**

```
method GetMessageUID(MessageNumber As %Integer,
    ByRef UniqueID As %String) as %Status {}
```

Returns, by reference, the UID of a message, given a message number. See the previous section for details on message numbers and UIDs. This method returns an error status if the message is currently marked for deletion.

GetMessageUIDArray()

Class Member

```
method GetMessageUIDArray(MessageNumber As %String = "",  
                          ByRef ListOfUniqueIDs As %ArrayOfDataTypes) as %Status {}
```

If given an empty string as the first argument, this method returns, by reference, an array of information about the messages in the mailbox (excluding any that are currently marked for deletion). Each element in this array contains the following information about one message:

Array Key	Array Item
Number of the message, within the mailbox in its current state. The first message is number 1, and so on. The message number of a given message is not guaranteed to be the same in all sessions.	Unique message identifier (UID), which is the permanent identifier of this message available in all sessions. UIDs are unique to each mailbox.

Or, given a message number, this method returns a one-element array that contains the UID of that message. In this case, the method returns an error status if the message is currently marked for deletion.

GetSizeOfMessages()

Class Member

```
method GetSizeOfMessages(MessageNumber As %String = "",  
                          ByRef ListOfSizes As %ArrayOfDataTypes) as %Status {}
```

If given an empty string as the first argument, this method returns, by reference, an array of information about the messages in the mailbox (excluding any that are currently marked for deletion). Each element in this array contains the following information about one message:

Array Key	Array Item
Number of the message, within the mailbox in its current state.	Size of this message, in bytes.

Or, given a message number, this method returns a one-element array that contains the size (in bytes) of that message. In this case, this method returns an error status if the message is currently marked for deletion.

6.10 See Also

- [Email Support in InterSystems IRIS](#)
- Using the Email Inbound Adapter (for interoperability productions)
- %Net.FetchMailProtocol (for examples and extensive comments)
- [Working with Received Email](#)

7

Working with Received Email

This page describes how you can work with an email message (`%Net.MailMessage`) that you have received.

7.1 Message Basics

After you retrieve an email message (`%Net.MailMessage`), you generally start by determining what kind of message it is and how to read it; that is, whether it is a multipart message and whether the parts are binary. In this step, you can use the `ContentType` property. Or you can use the `IsBinary`, `IsHTML`, and `IsMultiPart` properties, which indirectly provide the same information as `ContentType`.

If the message is a multipart message, each part is an instance of `%Net.MailMessagePart`.

7.2 Message Contents

Once you know the general message structure, use the following techniques to retrieve the contents:

- For a multipart message, use the `Parts` property, which is an array of the parts. `Parts.Count()` gives you the number of parts. The key for each part is an integer, starting with 1. Use the **GetAt()** method to retrieve a given part. A message part is an instance of `%Net.MailMessagePart`.

For information on the relationship of `%Net.MailMessage` and `%Net.MailMessagePart`, see [How InterSystems IRIS Represents MIME Email Messages](#).

- For a binary message (or message part), use the `BinaryData` property.
- For a text message (or message part), use the `TextData` property.
 - If `IsHTML` is 0, the `TextData` property is an ordinary text string.
 - If `IsHTML` is 1, the `TextData` property is an HTML text string.

Note that the email client that sends a message determines any wrapping in the message. The mail server has no control over this; nor does InterSystems IRIS.

7.3 Message Headers

The message itself and each part of the message has a set of headers.

The `%Net.MailMessage` class provides properties that give you the most commonly used headers.

- **To**—The list of email addresses to which this message was sent. This property is a standard InterSystems IRIS list; to work with it, you use the standard list methods: **Insert()**, **GetAt()**, **RemoveAt()**, **Count()**, and **Clear()**.
- **From**—The email address from which this message was sent.
- **Date**—The date of this message.
- **Subject**—A string containing the subject for this message.
- **Sender**—The actual sender of the message.
- **Cc**—The list of carbon copy addresses to which this message was sent.
- **Bcc**—The list of blind carbon copy addresses to which this message was sent. If the receiver of the message was on the blind carbon copy list, this property contains the address of the receiver; otherwise, it is null.

Both `%Net.MailMessage` and `%Net.MailMessagePart` provide the following properties, which give you other headers:

- **ContentType**—The `Content - Type` header of the message or message part.
- **ContentTransferEncoding**—The `Content - Transfer - Encoding` header of the message or message part.
- **Headers**—A multidimensional array that gives access to any additional headers.

Also, you can use the **GetAttribute()** method. Given a header name and an attribute, this method returns the value of that attribute.

7.4 Other Message Information

The `MessageSize` property indicates the total length of the message, apart from any attached email messages.

The following methods provide additional information about the message:

GetLocalDateTime()

Returns the date and time when the message was retrieved, converted to local time in **\$HOROLOG** format.

GetUTCDateTime()

Returns the date and time when the message was retrieved, converted to UTC in **\$HOROLOG** format.

GetUTCSeconds()

Returns the date and time when the message was retrieved, in seconds since 12/31/1840.

The following class methods are also available for time/date conversion:

HToSeconds()

A class method that converts a date/time in **\$HOROLOG** format to seconds since 12/31/1840.

SecondsToH()

A class method that converts seconds since 12/31/1840 to a date/time in **\$HOROLOG** format.

7.5 Example 1: ShowMsgInfo()

Given an instance of %Net.MailMessage, the following method writes information about the message to the current device:

Class Member

```
ClassMethod ShowMsgInfo(msg as %Net.MailMessage)
{
    Write "Message details *****" !
    Write "To (count): ", msg.To.Count(), !
    Write "From: ", msg.From, !
    Write "Cc (count): ", msg.Cc.Count(), !
    Write "Bcc (count): ", msg.Bcc.Count(), !
    Write "Date: ", msg.Date, !
    Write "Subject: ", msg.Subject, !
    Write "Sender: ", msg.Sender, !
    Write "IsMultipart: ", msg.IsMultiPart, !
    Write "Number of parts: ", msg.Parts.Count(), !
    Write "Number of headers: ", msg.Headers.Count(), !
    Write "IsBinary: ", msg.IsBinary, !
    Write "IsHTML: ", msg.IsHTML, !
    Write "TextData: ", msg.TextData.Read(), !
    Write "BinaryData: ", msg.BinaryData.Read(), !
}
```

This method produces output similar to the following:

```
Message details *****
To (count): 1
From: "XXX XXX" <XXX@XXX.com>
Cc (count): 0
Bcc (count): 0
Date: Fri, 16 Nov 2023 11:57:46 -0500
Subject: test 5
Sender:
IsMultipart: 0
Number of parts: 0
Number of headers: 16
IsBinary: 0
IsHTML: 0
TextData: This is test number 5, which is plain text.
BinaryData:
```

7.6 Example 2: ShowMsgPartInfo()

The following method writes information about a *part* of a message:

Class Member

```
ClassMethod ShowMsgPartInfo(msg as %Net.MailMessage, partno as %Integer)
{
    Set part=msg.Parts.GetAt(partno)
    Write "Message part details *****", !
    Write "Message part: ", partno, !
    Write "IsMultipart: ", part.IsMultiPart, !
    Write "Number of parts: ", part.Parts.Count(), !
    Write "Number of headers: ", part.Headers.Count(), !
    Write "IsBinary: ", part.IsBinary, !
    Write "IsHTML: ", part.IsHTML, !
    Write "TextData: ", part.TextData.Read(), !
    Write "BinaryData: ", part.BinaryData.Read(), !
}
```

This produces output similar to the following (given a different message than previously shown):

```
Message part details *****
Message part: 1
IsMultipart: 0
Number of parts: 0
Number of headers: 2
IsBinary: 0
IsHTML: 0
TextData: 1 test string
```

BinaryData:

7.7 Example 3: ShowMsgHeaders()

The following method writes information about the headers of a message; you could write a similar method that did the same for a message part.

Class Member

```
ClassMethod ShowMsgHeaders(msg as %Net.MailMessage)
{
    Set headers=msg.Headers
    Write "Number of headers: ", headers.Count(),!

    //iterate through the headers
    Set key=""
    For {
        Set value=headers.GetNext(.key)
        Quit:key=""
        Write "Header:",key,!
        Write "Value: ",value,!!
    }
}
```

This produces output similar to the following:

```
Number of headers: 16
Header: content-class
Value: urn:content-classes:message

Header: content-type
Value: multipart/alternative; boundary="----=_NextPart_001_01C8286D.D9A7F3B1"

Header: date
Value: Fri, 16 Nov 2023 11:29:24 -0500

Header: from
Value: "XXX XXX" <XXX@XXX.com>

Header: message-id
Value: <895A9EF10DBA1F46A2DDB3AAF061ECD501801E86@Exchange1_backup>

Header: mime-version
Value: 1.0

...
```

7.8 See Also

- [Email Support in InterSystems IRIS](#)
- [Retrieving Email Messages](#)
- [Creating and Writing MIME Messages](#)

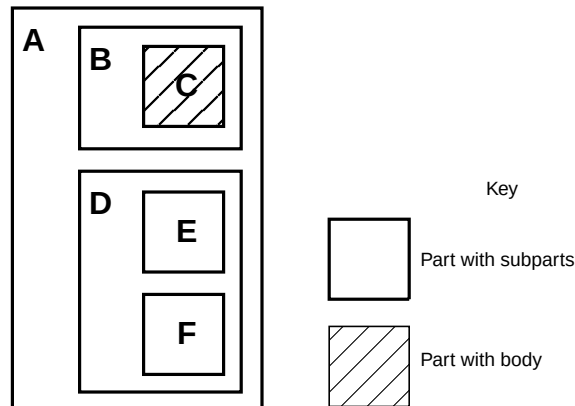
8

Creating, Writing, and Reading MIME Messages

InterSystems IRIS® data platform provides a class that you can use to create multipart MIME messages (%Net.MIMEPart). You use this class when you create attachments to add to SOAP messages; see [Creating Web Services and Web Clients](#). There are many other possible applications, such as [email](#) processing and [HTTP](#) multipart POST.

8.1 Overview

A document in MIME format is referred to as a *MIME part*. Each MIME part has headers and either contains a message body (either text or binary) or contains additional MIME parts. A MIME part that has a `MIME-Version` header can be used as a top-level document and is called a *MIME message*. The following figure shows an example:



In this example, E and F have additional subparts that are not shown.

To represent a MIME part, you use the %Net.MIMEPart class, which provides properties that you use to set the headers and contents of the part.

8.2 Creating MIME Parts

To create a MIME part, do the following:

1. Create an instance of `%Net.MIMEPart`.
2. Do one of the following:
 - Add a text or binary body. To do so, create an instance of a stream (either text or binary) and set the `Body` property of your MIME part equal to this stream. Use the standard stream interface to write data into this stream. Do not specify a value for the `Parts` property.
 - Add a list of MIME parts. To do so, create the MIME parts as described here and set the `Parts` property equal to a list of these parts. Do not specify a value for the `Body` property.
3. Optionally set headers as described in [Setting and Getting MIME Part Headers](#).

8.2.1 Setting and Getting MIME Part Headers

You can set values for and get values of the HTTP headers. The following properties of `%Net.MIMEPart` affect the MIME headers:

- `ContentType` — The [Internet media type](#) (MIME type) of the `Content - Type` header. This specifies the Internet media type of the `Body` data. For example: `"text/plain"`, `"text/html"`, `"image/jpeg"`, `"multipart/mixed"` and so on.
- `ContentCharset` — The `charset` part of the `Content - Type` header. If you set this, you must *first* set the `ContentType` property. For each MIME part that contains a text body, be sure to set the `ContentCharset` property appropriately to indicate the character set used in the body. This property should declare the character set that is already used, since `%Net.MIMEPart` does not perform any conversion.
- `ContentId` — The normalized `Content - ID` header, without the angle brackets (`<>`) and any leading and trailing spaces.
- `ContentLocation` — The normalized `Content - Location` header, without any leading and trailing spaces.
- `ContentTransferEncoding` — The `Content - Transfer - Encoding` header. This property can be one of the following: `"base64"` `"quoted-printable"` `"7bit"` `"8bit"`

There is no default value.

Important: Note that if the content is `"base64"` encoded, it cannot contain any Unicode characters. If the content you wish to send includes Unicode characters, then make sure to use `$ZCONVERT` to convert the content to UTF-8, and then base-64 encode it. For example:

ObjectScript

```
set BinaryText=$ZCONVERT(UnicodeText,"0","UTF8")
set Base64Encoded=$system.Encryption.Base64Encode(BinaryText)
```

The recipient must use the reverse process to decode the text:

ObjectScript

```
set BinaryText=$system.Encryption.Base64Decode(Base64Encoded)
set UnicodeText=$ZCONVERT(BinaryText,"I","UTF8")
```

The `%Net.MIMEPart` class provides general methods that you can use to manage the MIME headers:

- **GetHeader()** returns the value of a header.
- **NextHeader()** gets the next header.
- **SetHeader()** sets the value of a header. Typically you use this to set nonstandard headers.
- **RemoveHeader()** removes a header.

For complete method signatures and other details, see the class documentation for `%Net.MIMEPart`.

8.2.2 Specifying an Optional Message Boundary Value

By default, message boundaries are generated automatically. You can specify the message boundary, if needed. To do so, specify a value for the `Boundary` property. Be sure to use a string that is extremely unlikely to be used in any of the message parts.

8.3 Writing MIME Messages

To write MIME messages, use `%Net.MIMEWriter` as follows:

1. Create an instance of the `%Net.MIMEWriter` class.
2. Optionally specify an output destination. To do so, use one of the following methods of your writer instance: **OutputToDevice()** (the default), **OutputToFile()**, or **OutputToStream()**.
3. Call methods of your writer to write output as needed:
 - Given a header name and value, **WriteHeader()** writes that header.
 - Given an instance of `%Net.MIMEPart`, **WriteMIMEBody()** writes the message body, which can have multiple parts.

If the message is multipart, this method does not write any headers; it is your responsibility to write them. If the message is not multipart, however, the method does write the headers.
 - Given an instance of `%Net.MIMEPart`, **WriteMIMEMessage()** writes the MIME message, including all headers.

For single-part messages, **WriteMIMEBody()** and **WriteMIMEMessage()** produce the same output.

For complete method signatures and other details, see the class documentation for `%Net.MIMEPart`.

8.3.1 Example: WriteMIMEMessage()

The following example demonstrates the use of **WriteMIMEMessage()**:

Class Member

```
ClassMethod WriteMIMEMessage(text As %String,header as %String) as %Status
{
  Set msg=##class(%Net.MIMEPart).%New()
  Set msg.Body=##class(%GlobalCharacterStream).%New()
  Do msg.Body.Write(text)

  //specify some headers
  Set msg.ContentType="text/html"
  Set msg.ContentCharset="us-ascii"
  Do msg.SetHeader("Custom-header",header)

  //create MIME writer; write MIME message
  Set writer=##class(%Net.MIMEWriter).%New()
  Set status=writer.WriteMIMEMessage(msg)

  If $$$ISERR(status) do $system.Status.DisplayError(status)
  Quit $$$OK
}
```

The following Terminal session shows this method in use:

```
GNET> Set text = "message text"
GNET> Set header="my header value"
GNET> Do ##class(GNET.MIME).WriteMIMEMessage(text,header)
CONTENT-TYPE: text/html
Custom-header: my header value
message text
GNET>
```

8.4 Reading MIME Messages

To read MIME messages, use %Net.MIMEReader, as follows:

1. Create an instance of the %Net.MIMEReader class.
2. Specify the source of input. To do so, use one of the following methods of your reader instance: **OpenFile()** or **OpenStream()**.
3. Call the **ReadMIMEMessage()** method of your reader instance. This method returns an instance of %Net.MIMEPart by reference as the first argument. It returns a status, which you should check.

For complete method signatures and other details, see the class documentation for %Net.MIMEPart.

9

Using FTP

InterSystems IRIS® data platform provides a class, `%Net.FtpSession`, that you can use to establish a session with an FTP server from within InterSystems IRIS.

Note: You can use this class with FTP or FTPS (FTP with SSL/TLS), but if your use case is specifically SFTP (SSH FTP), then use `%Net.SSH.Session` instead; see [Using SSH](#).

9.1 Establishing an FTP Session

To establish an FTP session, do the following:

1. Create an instance of `%Net.FtpSession`.
2. Optionally set properties of this instance in order to control the general behavior of the session:
 - `Timeout` specifies how long to wait for a reply from the FTP server, in seconds.
 - `SSLConfiguration` specifies the activated SSL/TLS configuration to use for the connection, if any. Use this if the FTP server uses https.

For information on creating and managing SSL/TLS configurations, see [InterSystems TLS Guide](#). The SSL/TLS configuration includes an option called **Configuration Name**; this is the string to use in this setting.
 - `TranslateTable` specifies the [translation table](#) to use when reading the contents of a file or writing the contents of a file.

To find the name of the table for a given character set, use the `%Net.Charset` class.
 - `UsePASV` enables PASV mode.
 - `SSLCheckServerIdentity` applies when the FTP server uses https. By default, when an instance of `%Net.FtpSession` connects to a SSL/TLS server, it checks whether the certificate server name matches the DNS name used to connect to the server. If these names do not match, the connection is not permitted. This default behavior prevents man in the middle attacks and is described in [RFC 2818](#), section 3.1; also see [RFC 2595](#), section 2.4.

To disable this check, set the `SSLCheckServerIdentity` property to 0.
 - `SSLUseSessionResumption` if true, specifies that when making the SSL connection for the data channel, reuse session parameters from the command channel. This feature requires OpenSSL v1.1.x+. If you set the flag on an instance using OpenSSL v1.0.x, it is ignored.
3. Call the **Connect()** method to connect to a specific FTP server.

4. Call the **Ascii()** or **Binary()** method to set the transfer mode to ASCII mode or binary mode, respectively. To see the current transfer mode, check the value of the **Type** property of your instance.

Note: Each method of `%Net.FtpSession` returns a status, which you should check. The methods also set the values of properties that provide useful information on the state of the session:

- **Connected** is true if you are currently connected, and is false otherwise.
- **ReturnCode** contains the return code from the last communication with the FTP server.
- **ReturnMessage** contains the return message from the last communication with the FTP server.

The **Status()** method returns (by reference) the status of the FTP server.

For details, see the class documentation for `%Net.FtpSession`.

9.1.1 Translate Table for Commands

`%Net.FtpSession` uses the technique described in [RFC 2640](#) to automatically handle character set translation when looking at filenames and pathnames on an FTP server. When an instance of `%Net.FtpSession` connects to an FTP server, it uses the FEAT message to determine whether the server UTF-8 characters. If so, it switches the command channel communication to UTF-8 so that all filenames and pathnames will be correctly translated to and from UTF-8.

If the server does not support the FEAT command or does not report that it supports UTF-8, the `%Net.FtpSession` instance uses RAW mode and reads or writes the raw bytes.

In rare cases, if you need to specify the [translation table](#) to use, set the **CommandTranslateTable** property of the `%Net.FtpSession` instance. It should not be generally necessary to use this property.

9.2 FTP File and System Methods

Once you establish an FTP session, call methods of your session instance to perform FTP tasks. `%Net.FtpSession` provides the following methods for reading and writing files:

Delete()

Deletes a file.

Retrieve()

Copies a file from the FTP server into an InterSystems IRIS stream and returns the stream by reference. To work with this stream, use the standard stream methods: **Write()**, **WriteLine()**, **Read()**, **ReadLine()**, **Rewind()**, **MoveToEnd()**, and **Clear()**. You can also use the **Size** property of the stream.

RetryRetrieve()

Allows you to continue retrieving a file, given a stream created by a previous use of **Retrieve()**.

Store()

Writes the contents of an InterSystems IRIS stream into a file on the FTP server.

StoreFiles()

Given a local directory, and a wildcard mask, this method writes multiple files to that directory. Note that this method ignores directories, and uses the current transfer mode (binary or ASCII), which means that you cannot upload a mixed set of binary and ASCII files in one call.

Append()

Appends the contents of a stream to the end of the specified file.

Rename()

Renames a file.

In addition, %Net.FtpSession provides methods for navigating and modifying the file system on the FTP server:

GetDirectory(), **SetDirectory()**, **SetToParentDirectory()**, and **MakeDirectory()**.

To examine the contents of the file system, use the **List()** or **NameList()** methods.

- **List()** creates a stream that contains a list of all the files whose names match a given pattern and returns this stream by reference.
- **NameList()** creates an array of filenames and returns this array by reference.

You can also use the **ChangeUser()** method to change to a different user; this is faster than logging out and logging in again. Use the **Logout()** method to log out.

The **System()** method returns (by reference) information about the type of computer that is hosting the FTP server.

The **Size()** and **MDTM()** methods return a file's size and its modification time, respectively.

Use the generic **sendCommand()** method to send a command to the FTP server and to read the response. This method may be used to send commands that are not explicitly supported in %Net.FtpSession.

For details, see the class documentation for %Net.FtpSession.

9.3 Using a Linked Stream to Upload Large Files

If you have a large file to upload, consider using the **LinkToFile()** method of the stream interface. That is, instead of creating a stream and reading the file into it, create a stream and link it to the file. Use this linked stream when you call the **Store()** method of %Net.FtpSession.

For example:

Class Member

```
Method SendLargeFile(ftp As %Net.FtpSession, dir As %String, filename As %String)
{
    Set filestream=##class(%FileBinaryStream).%New()
    Set sc=filestream.LinkToFile(dir_filename)
    If $$$ISERR(sc) {do $System.Status.DisplayError(sc) quit }

    //Uploaded file will have same name as the original
    Set newname=filename

    Set sc=ftp.Store(newname,filestream)
    If $$$ISERR(sc) {do $System.Status.DisplayError(sc) quit }
}
```

9.4 Customizing Callbacks Issued by the FTP Server

You can customize the callbacks generated by the FTP server. By doing so, you can for example, give the user an indication that the server is still working on a large transfer, or allow the user to abort the transfer.

To customize the FTP callbacks:

1. Create a subclass of `%Net.FtpCallback`.
2. In this subclass, implement the **RetrieveCallback()** method, which is called at regular intervals when receiving data from the FTP server.
3. Also implement the **StoreCallback()** method, which is called at regular intervals when writing data to the FTP server.
4. When you create an FTP session (as described in [Establishing an FTP Session](#)), set the `Callback` property equal to your subclass of `%Net.FtpCallback`.

For details, see the class documentation for `%Net.FtpCallback`.

9.5 See Also

- [Using SSH](#)

10

Using SSH

The %Net.SSH package provides support for SSH ([Secure Shell](#)) communications. This topic briefly introduces the classes in this package.

CAUTION: OpenSSL 3.0 introduces a new concept of providers, which packages groups of algorithm implementations. One provider is the *Legacy provider*. Neither OpenSSL nor InterSystems IRIS® data platform loads the Legacy provider by default.

InterSystems recommends that you do not use the Legacy provider's algorithms; a complete list of these algorithms can be found in the [OpenSSL documentation](#). InterSystems cannot guarantee that any legacy algorithm is compatible with InterSystems IRIS.

10.1 Creating an SSH Session

%Net.SSH.Session represents an SSH session. To use this class:

1. Create an instance of the class.
2. Use the **Connect()** instance method to connect to a server.
3. Use either **AuthenticateWithKeyPair()**, **AuthenticateWithUsername()**, or **AuthenticateWithKeyboardInteractive()** to authenticate yourself to the server. For details, see the class reference for %Net.SSH.Session.
4. Use additional methods of %Net.SSH.Session to perform SCP (Secure Copy) operations of single files to and from the remote system, execute remote commands, tunnel TCP traffic, or perform SFTP operations. See the class reference for %Net.SSH.Session.

For example, use **OpenSFTP** to use the session for SFTP operations. This method returns, by reference, an instance of %Net.SSH.SFTP that you can use for SFTP operations. See the example in the next section.

- Important:**
- For information on the supported platforms where you can use these classes, see the class reference for %Net.SSH.Session and %Net.SSH.SFTP.
 - SSH connections that use OpenSSL 3.0 do not support Blowfish or CAST as the cipher algorithm. If you attempt to establish such a connection, the attempt will fail.

10.2 Supported Key Types and Hostkey Algorithms

To establish an SSH connection between a client and server, the two parties must prove to each other their identities. To do this, each party has two keys: a private key and public key, called a public/private key pair. A key pair is defined by its type and format:

- **Type:** The cryptographic algorithm with which the key pair was constructed.

The key pair type is written in the public (. pub) key file. For example, an ED25519 key pair shows the `ssh-ed25519` key type in its public key:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK+fbizguJPey1+7q2uNMvn8ku9Vd4MEt43jWfng8DAs user@example
```

- **Format:** The way in which the keys are encoded. This typically differs between the public and private keys. The public key is always in OpenSSH format, which is the format used for entries in `.ssh/authorized_keys` and `.ssh/known_hosts` on the server and client, respectively. The private key format can vary, though the most common are OpenSSH and PEM.

Each key type has an associated hostkey algorithm, which determines how the key pair is used in an SSH connection. To use a key pair with InterSystems IRIS, its type's associated hostkey algorithm must be supported by both InterSystems IRIS and the target server.

With the exception of RSA type key pairs, a key type and its associated hostkey algorithm share the same name. For example, `ssh-ed25519` type keys are associated with the hostkey algorithm `ssh-ed25519`, so if you want to use this key type, both InterSystems IRIS and the target server must support the `ssh-ed25519` hostkey algorithm.

The exception to this naming rule is RSA (type `ssh-rsa`) keys, which ["are not dependent on the choice of hash function."](#) This means that `ssh-rsa` type keys can be used with any of the following algorithms:

- `ssh-rsa` (deprecated for using SHA1)
- `rsa-sha2-256`
- `rsa-sha2-512`

To view the hostkey algorithms (among others) supported by your version of InterSystems IRIS, you can use **GetSupportedAlgorithms()**. For example:

```
set ses = ##class(%Net.SSH.Session).%New()  
set sc = ses.GetSupportedAlgorithms(.algos)
```

10.3 Example: Listing Files via SFTP

The following method shows how you can write a list of the files on a server, via SFTP:

Class Member

```
Method SFTPDDir(ftpserver, username, password) As %Status
{
    set ssh = ##class(%Net.SSH.Session).%New()
    set status = ssh.Connect(ftpserver)
    set status = ssh.AuthenticateWithUsername(username,password)
    //open an SFTP session and get that returned by reference
    set status = ssh.OpenSFTP(.sftp)
    //get a list of files
    set status = sftp.Dir(".",.files)
    set i=$ORDER(files(""))
    while i="" {
        write $listget(files(i),1),!
        set i=$ORDER(files(i))
    }
    quit $$$OK
}
```

10.4 Example: Authenticate with Keyboard Interactive

The following Terminal session demonstrates how to use **AuthenticateWithKeyboardInteractive()**:

Terminal

```
set host="192.168.2.100"
set lambda="(u,i,p,f,c) quit $listbuild(c("password"))"
set context("password")="fountain"
set sess=##class(%Net.SSH.Session).%New()
s status=sess.Connect(host)
set status=sess.AuthenticateWithKeyboardInteractive("root",lambda,.context)
```

10.5 Example: Authenticate with Key Pair

The following example demonstrates how to use **AuthenticateWithKeyPair()**:

Terminal

```
set host="example.com"
set username="Bob"
set publickeyfile="/Users/Bob/.ssh/id_ecdsa.pub"
set privatekeyfile="/Users/Bob/.ssh/id_ecdsa"

Set sess = ##class(%Net.SSH.Session).%New()
Set status = sess.Connect(host, 22)
Set status = sess.AuthenticateWithKeyPair(username, publickeyfile, privatekeyfile)
```

10.6 Example: Execute a Remote Command

The following examples demonstrate how to use **Execute()**:

ObjectScript

```
// %Net.SSH.Session.Execute
// Specify the initial SSH connection information
Set host="192.168.2.37"
Set username = "SSHUser"
Set password = "SSHPassword"
// Set up the initial SSH connection
// localhost <--SSH--> 192.168.2.37
Set sshSession = ##class(%Net.SSH.Session).%New()
Set statusConnection = sshSession.Connect(host)
Set statusConnection = sshSession.AuthenticateWithUsername(username,password)
// Specify the command to execute on the remote host
Set command = "uname -a"
// Execute the command over the SSH connection
// tDevice is a pass-by-reference value to store the raw data passing through the connection
Set statusConnection = sshSession.Execute(command,.tDevice)
```

10.7 Example: Port Forward

The following examples demonstrate how to use **ForwardPort()**

ObjectScript

```
// %Net.SSH.Session.ForwardPort
// Specify the initial SSH connection information
Set host="192.168.2.37"
Set username = "SSHUser"
Set password = "SSHPassword"
// Set up the initial SSH connection
// localhost <--SSH--> 192.168.2.37
Set sshSession = ##class(%Net.SSH.Session).%New()
Set statusConnection = sshSession.Connect(host)
Set statusConnection = sshSession.AuthenticateWithUsername(username,password)
// Specify the remote port forward information
Set remotehost = "192.168.2.100"
Set remoteport = 80
// Forward traffic via the SSH connection to a remote host:port
// localhost <--SSH--> 192.168.2.37 --SSHPortFwd--> 192.168.2.100:80
// tDevice is a pass-by-reference value to store the raw data passing through the connection
Set statusConnection = sshSession.ForwardPort(remotehost,remoteport,.tDevice)
```

10.8 See Also

- [Using and debugging %Net.SSH.Session for SSH connections](#) (Developer Community)

11

Sending and Receiving IBM WebSphere MQ Messages

InterSystems IRIS® data platform provides an interface to IBM WebSphere MQ, which you can use to exchange messages between InterSystems IRIS and the message queues of IBM WebSphere MQ. To use this interface, you must have access to an IBM WebSphere MQ server, and the IBM WebSphere MQ client must be running on the same machine as InterSystems IRIS.

The interface consists of the `%Net.MQSend` and `%Net.MQRecv` classes, which are both subclasses of `%Net.abstractMQ`. These classes use a dynamic-link library that is automatically installed by InterSystems IRIS on all suitable platforms. (This is `MQInterface.dll` on Windows; the file extension is different for other platforms.) In turn, the InterSystems IRIS dynamic-link library requires IBM WebSphere MQ dynamic-link libraries.

The interface supports sending and receiving only text data, not binary data.

To use IBM WebSphere MQ, you will need the formal documentation for this product. Also, for additional information on the InterSystems IRIS interface to IBM WebSphere MQ, see the class reference for `%Net.abstractMQ`.

11.1 Overview

In general, to use the InterSystems IRIS interface to IBM WebSphere MQ, you do the following:

1. Make sure that you have access to IBM WebSphere MQ v7.x or higher. Specifically:
 - The IBM WebSphere MQ client must be installed on the same machine as InterSystems IRIS. Note that the installer updates the *PATH* environment variable and adds other system variables as needed.
 - Make sure that you have rebooted the machine after installing the client, so that InterSystems IRIS is aware of the client.
 - The client must have access to an IBM WebSphere MQ server.
 - The username under which you will access the server must have permission to use the queue managers and the queues that you plan to use.
2. Create a new instance of `%Net.MQSend` or `%Net.MQRecv`, depending on whether you are going to send or receive messages.
3. Connect to an IBM WebSphere MQ server. When you do so, you provide the following information:
 - The name of a queue manager.

- The name of a queue to use.
- The channel by which to communicate with that queue. You specify a channel name, a transport mechanism, and the IP address and port of the IBM WebSphere MQ server.

You can also provide a name and password if you are using the authentication feature of IBM WebSphere MQ.

4. Invoke the appropriate methods of %Net.MQSend or %Net.MQRecv to send or receive messages.

Note: To use IBM Websphere MQ on 64-bit Linux platforms, you must set the LD_LIBRARY_PATH to include the location of the MQ libraries. Because the path must be set for any InterSystems IRIS process that uses the MQ interface, it must be set prior to starting InterSystems IRIS if running background processes, and set in any UNIX® terminal prior to running **iris terminal**.

11.1.1 Getting Error Codes

The methods of %Net.MQSend and %Net.MQRecv return either 1 if successful or 0 if unsuccessful. In the case of an error, call the **%GetLastError()** method, which returns the last reason code given by IBM WebSphere MQ. For information on the reason codes, see the formal IBM documentation.

11.2 Creating a Connection Object

Before you can send or receive messages via IBM WebSphere MQ, you must create a *connection object*, an object that establishes a connection to a queue manager, opens a channel, and opens a queue for use. There are two ways you can do this:

- You can use the [%Init method](#), which takes arguments that specify all the needed information.
- You can use the [%Connect method](#) after first setting properties that specify all the needed information.

11.2.1 Using the %Init() Method

To use the %Init() method to create a connection object:

1. Create an instance of %Net.MQSend (if you are going to send messages) or %Net.MQRecv (if you are going to receive messages). This topic refers to this instance as the connection object.

Note: If you receive the <DYNAMIC LIBRARY LOAD> error, a dynamic-link library is missing, and the messages.log file (in the system manager's directory) has more details.

2. Call the **%Init()** method of the connection object. This method takes the following arguments, in order.
 - a. (Required) A string that specifies the queue name; this should be a valid queue for the specified queue manager.
 - b. A string that specifies the queue manager; this should be a valid queue manager on the IBM WebSphere MQ server.

If you omit this argument, the system uses the default queue manager, as configured in IBM WebSphere MQ. Or, if IBM WebSphere MQ has been configured so that the queue manager is determined by the queue name, the system uses the queue manager that is appropriate for the given queue name.

- c. A string that specifies the specification for the channel, in the following form:

"channel_name/transport/hostname(port)"

Here *channel_name* is the name of the channel to use, *transport* is the transport used by the channel, *hostname* is the server name (or IP address) that is running the IBM WebSphere MQ server, and *port* is the port that this channel should use.

Transport can be one of the following: TCP, LU62, NETBIOS, SPX

For example:

```
"CHAN_1/TCP/rodan(1401)"
```

```
"CHAN_1/TCP/127.0.0.1(1401)"
```

If the MQ Server supports this, you can use a comma-separated list of host names with ports. For example:
CHAN_1/TCP/rodan(1401), rodan2(1401)

If you omit this argument, the system uses the default channel specification, as configured in IBM WebSphere MQ. Or, if the system has been configured so that the channel is determined by the queue name, the system uses the channel that is appropriate for the given queue name.

- d. An optional string that specifies the log file to write error messages to. The default is that no logging occurs.
3. Check the value returned by the **%Init()** method. If the method returns 1, the connection was established successfully, and you can use the connection object to either send or receive messages (depending on the class you are using). See [Getting Error Codes](#).

11.2.2 Using the %Connect() Method

In some cases, you might prefer to specify all the details of the connection individually. To do so, you use the **%Connect()** method, as follows:

1. Create an instance of %Net.MQSend (if you are going to send messages) or %Net.MQRecv (if you are going to receive messages). As noted previously, this topic refers to this instance as the connection object.

Note: If you receive the <DYNAMIC LIBRARY LOAD> error, a dynamic-link library is missing, and the messages.log file (in the system manager's directory) has more details.

2. Set the following properties of the connection object:

- QName — (Required) Specifies the queue name; this should be a valid queue for the specified queue manager.
- QMgr — Specifies the queue manager to use; this should be a valid queue manager on the IBM WebSphere MQ server.

If you omit this argument, the system uses the default queue manager, as configured in IBM WebSphere MQ. Or, if IBM WebSphere MQ has been configured so that the queue manager is determined by the queue name, the system uses the queue manager that is appropriate for the given queue name.

3. Optionally specify the channel to use by setting the following properties of the connection object:

- Connection — Specifies the host and port of the IBM WebSphere MQ server. For example: "127.0.0.1:1401".
- Channel — Specifies the name of the channel to use. This must be a valid channel on the IBM WebSphere MQ server.
- Transport — Specifies the transport used by the channel. This property can be one of the following: "TCP", "LU62", "NETBIOS", "SPX"

If you omit these arguments, the system uses the default channel specification, as configured in IBM WebSphere MQ. Or, if the system has been configured so that the channel is determined by the queue name, the system uses the channel that is appropriate for the given queue name.

4. Call the **%ErrLog()** method of the connection object. This method takes one argument, the name of the log file to use for this connection object.
5. Check the value returned by the **%ErrLog()** method. See [Getting Error Codes](#).
6. Call the **%Connect()** method of the connection object.
7. Check the value returned by the **%Connect()** method. If the method returns 1, the connection was established successfully, and you can use the connection object to either send or receive messages (depending on the class you are using). See [Getting Error Codes](#).

11.3 Specifying the Character Set (CCSID)

To set the character set used for message conversions, call the **%SetCharSet()** method of your connection object. Specify an integer Coded Character Set ID (CCSID) as used in IBM WebSphere MQ.

- If you are sending messages, this should be the character set of those messages. If you do not specify a character set, the MQ system assumes the messages use the default character set specified for the MQ client.
- If you are retrieving messages, this is the character set to which those messages will be translated.

To get the CCSID that is currently being used, call the **%CharSet()** method. This method returns the CCSID by reference and it returns 1 or 0 to indicate whether it was successful; see [Getting Error Codes](#).

For information on the CCSID that corresponds to a given character set, see the formal IBM documentation.

11.4 Specifying Other Message Options

To specify message descriptor options, optionally set the following properties of your connection object:

- `AppIdentityData` specifies the Application Identity message descriptor option.
- `PutAppType` specifies the Put Application Type message descriptor option.

11.5 Sending Messages

To send messages, do the following:

1. Create a connection object as described in [Creating a Connection Object](#). In this case, create an instance of `%Net.MQSend`. The connection object has a message queue to which you can send messages.
2. Call the following methods, as needed:
 - **%Put()** — Given a string, this method writes that string to the message queue.
 - **%PutStream()** — Given an initialized file character stream, this method writes that string to the message queue. Note that you must set the `Filename` property of the stream in order to initialize it. Binary streams are not supported.

- **%SetMsgId()** — Given a string, this method uses that string as the message ID for the next message that is sent.
3. Check the value returned by the method you called. See [Getting Error Codes](#).
 4. When you are done retrieving messages, call the **%Close()** method of the connection object to release the handle to the dynamic-link library.

11.5.1 Example 1: SendString()

The following class method sends a simple string message to the queue `mqtest`, using queue manager `QM_antigua`, and a queue channel named `S_antigua`. The channel uses TCP transport, and the IBM WebSphere MQ server is running on a machine called `antigua` and is listening on port 1401.

Class Member

```
///Method returns reason code from IBM WebSphere MQ
ClassMethod SendString() As %Integer
{
  Set send=##class(%Net.MQSend).%New()
  Set queue="mqtest"
  Set qm="QM_antigua"
  Set chan="S_antigua/TCP/antigua(1414)"
  Set logfile="c:\mq-send-log.txt"

  Set check=send.%Init(queue,qm,chan,logfile)
  If 'check Quit send.%GetLastError()

  //send a unique message
  Set check=send.%Put("This is a test message "_$h)

  If 'check Quit send.%GetLastError()
  Quit check
}
```

11.5.2 Example 2: SendCharacterStream()

The following class method sends the contents of a file character stream. It uses the same queue used in the previous example:

Class Member

```
///Method returns reason code from IBM WebSphere MQ
ClassMethod SendCharacterStream() As %Integer
{
  Set send=##class(%Net.MQSend).%New()
  Set queue="mqtest"
  Set qm="QM_antigua"
  Set chan="S_antigua/TCP/antigua(1414)"
  Set logfile="c:\mq-send-log.txt"

  Set check=send.%Init(queue,qm,chan,logfile)
  If 'check Quit send.%GetLastError()

  //initialize the stream and tell it what file to use
  Set longmsg=##class(%FileCharacterStream).%New()
  Set longmsg.Filename="c:\input-sample.txt"

  Set check=send.%PutStream(longmsg)

  If 'check Quit send.%GetLastError()
  Quit check
}
```

11.5.3 Example 3: Sending a Message from the Terminal

The following example shows a Terminal session that sends a message to an IBM WebSphere MQ queue. This works only on a machine that has been configured with the IBM WebSphere MQ client.

ObjectScript

```
Set MySendQ = ##class(%Net.MQSend).%New()  
Do MySendQ.%Init("Q_1", "QM_1", "QC_1/TCP/127.0.0.1(1401)", "C:\mq.log")  
Do MySendQ.%Put("Hello from tester")  
Set MyRecvQ = ##class(%Net.MQRecv).%New()  
Do MyRecvQ.%Init("Q_1", "QM_1", "QC_1", "C:\mq.log")  
Do MyRecvQ.%Get(.msg, 10000)  
Write msg, !
```

Also see the preceding sections for other examples.

11.6 Retrieving Messages

To retrieve messages, do the following:

1. Create a connection object as described in [Creating a Connection Object](#). In this case, create an instance of %Net.MQRecv. The connection object has a message queue from which you can retrieve messages.
2. Call the following methods, as needed:
 - **%Get()** — Returns a string message by reference as the first argument.
 - **%GetStream()** — Given an initialized file character stream, this method retrieves a message from the queue and places it into the file associated with that stream. Note that you must set the Filename property of the stream in order to initialize it. Binary streams are not supported.

For both methods, the second argument is the timeout, in milliseconds; this controls the time used to contact the server. The default timeout is 0.

3. Check the value returned by the method you called. See [Getting Error Codes](#). Remember that IBM WebSphere MQ returns 2033 when the queue is empty.
4. When you are done retrieving messages, call the **%Close()** method of the connection object to release the handle to the dynamic-link library.

11.6.1 Example 1: ReceiveString()

The following class method retrieves a message from the mqtest queue.

Class Member

```
///Method returns string or null or error message  
ClassMethod ReceiveString() As %String  
{  
  Set recv=##class(%Net.MQRecv).%New()  
  Set queue="mqtest"  
  Set qm="QM_antigua"  
  Set chan="S_antigua/TCP/antigua(1414)"
```

```

Set logfile="c:\mq-recv-log.txt"

Set check=recv.%Init(queue,qm,chan,logfile)
If 'check Quit recv.%GetLastError()

Set check=recv.%Get(.msg)
If 'check {
    Set reasoncode=recv.%GetLastError()
    If reasoncode=2033 Quit ""
    Quit "ERROR: " _reasoncode
}

Quit msg
}

```

11.6.2 Example 2: ReceiveCharacterStream()

The following method can retrieve a longer message because it uses **%GetStream()**:

Class Member

```

/// Method returns reason code from IBM WebSphere MQ
ClassMethod ReceiveCharacterStream() As %Integer
{
    Set recv=##class(%Net.MQRecv).%New()
    Set queue="mqtest"
    Set qm="QM_antigua"
    Set chan="S_antigua/TCP/antigua(1414)"
    Set logfile="c:\mq-recv-log.txt"

    Set check=recv.%Init(queue,qm,chan,logfile)
    If 'check Quit recv.%GetLastError()

    //initialize the stream and tell it what file to use
    //make sure filename is unique we can tell what we received
    Set longmsg=##class(%FileCharacterStream).%New()
    Set longmsg.Filename="c:\mq-received"_$h_.txt"

    Set check=recv.%GetStream(longmsg)

    If 'check Quit recv.%GetLastError()
    Quit check
}

```

11.7 Updating Message Information

The **%Net.MQSend** and **%Net.MQRecv** classes also provide the following methods:

%CorId()

Updates (by reference) the Correlation Id for the last message read.

%ReplyQMGrName()

Updates (by reference) the reply queue manager name for the last message read.

%ReplyQName()

Updates (by reference) the reply queue name for the last message read.

11.8 Troubleshooting

If you encounter problems when using the InterSystems IRIS interface to IBM WebSphere MQ, you should first determine whether the client is correctly installed and can communicate with the server. To perform such a test, you can use sample programs that are provided by IBM WebSphere MQ. The executables are in the bin directory of the IBM WebSphere MQ client.

The following steps describe how to use these sample programs on Windows. The details may be different on other operating systems; consult the IBM documentation and check the names of the files present in your client.

1. Create an environment variable called *MQSERVER*. Its value should be of the form *channel_name/transport/server*, where *channel_name* is the name of the channel to use, *transport* is a string that indicates the transport to use, and *server* is the name of the server. For example: *S_antigua/TCP/antigua*
2. At the command line, enter the following command:

```
amqsputc queue_name queue_manager_name
```

where *queue_name* is the name of the queue to use and *queue_manager_name* is the name of the queue manager. For example:

```
amqsputc mqtest QM_antigua
```

If the *amqsputc* command is unrecognized, make sure that the *PATH* environment variable has been updated to include the bin directory of the IBM WebSphere MQ client.

In case of other errors, consult the IBM documentation.

3. You should see a couple of lines like the following:

```
Sample AMQSPUT0 start  
target queue is mqtest
```

4. Now you can send messages. Simply type each message and press Enter after each message. For example:

```
sample message 1  
sample message 2
```

5. When you are done sending messages, press Enter twice. You will then see a line like the following:

```
Sample AMQSPUT0 end
```

6. To complete this test, we will retrieve the messages you sent to the queue. Type the following command at the command line:

```
amqsgetc queue_name queue_manager_name
```

where *queue_name* is the name of the queue to use and *queue_manager_name* is the name of the queue manager. For example:

```
amqsgetc mqtest QM_antigua
```

7. You should then see a start line, followed by the messages that you sent previously, as follows:

```
Sample AMQSGET0 start  
message <sample message 1>  
message <sample message 2>
```

8. This sample program waits briefly to receive any other messages and then displays the following:

```
no more messages  
Sample AMQSGE0 end
```

If the test fails, consult the IBM documentation. Possible causes of problems include the following:

- Security issues
- Queue is not defined correctly
- Queue manager is not started

